Week 4 — From Transistors to RTL and Synthesis

Format: seven daily sections collected into one weekly module

Suggested workload: approximately 21 focused hours, adjustable to mastery

> A register-transfer description is not a bag of transistors. It is a behavioral and structural specification that tools can simulate, elaborate, optimize, and map—provided its timing and hardware meaning are clear.

This hardware depth serves the end-to-end systems goal. It explains what exists beneath ISA-visible registers, instructions, exceptions, and memory accesses before the course returns to CPU microarchitecture, MMU/privilege, and ToyOS. It is **necessary depth, not a divergent specialization** in professional ASIC or physical design.

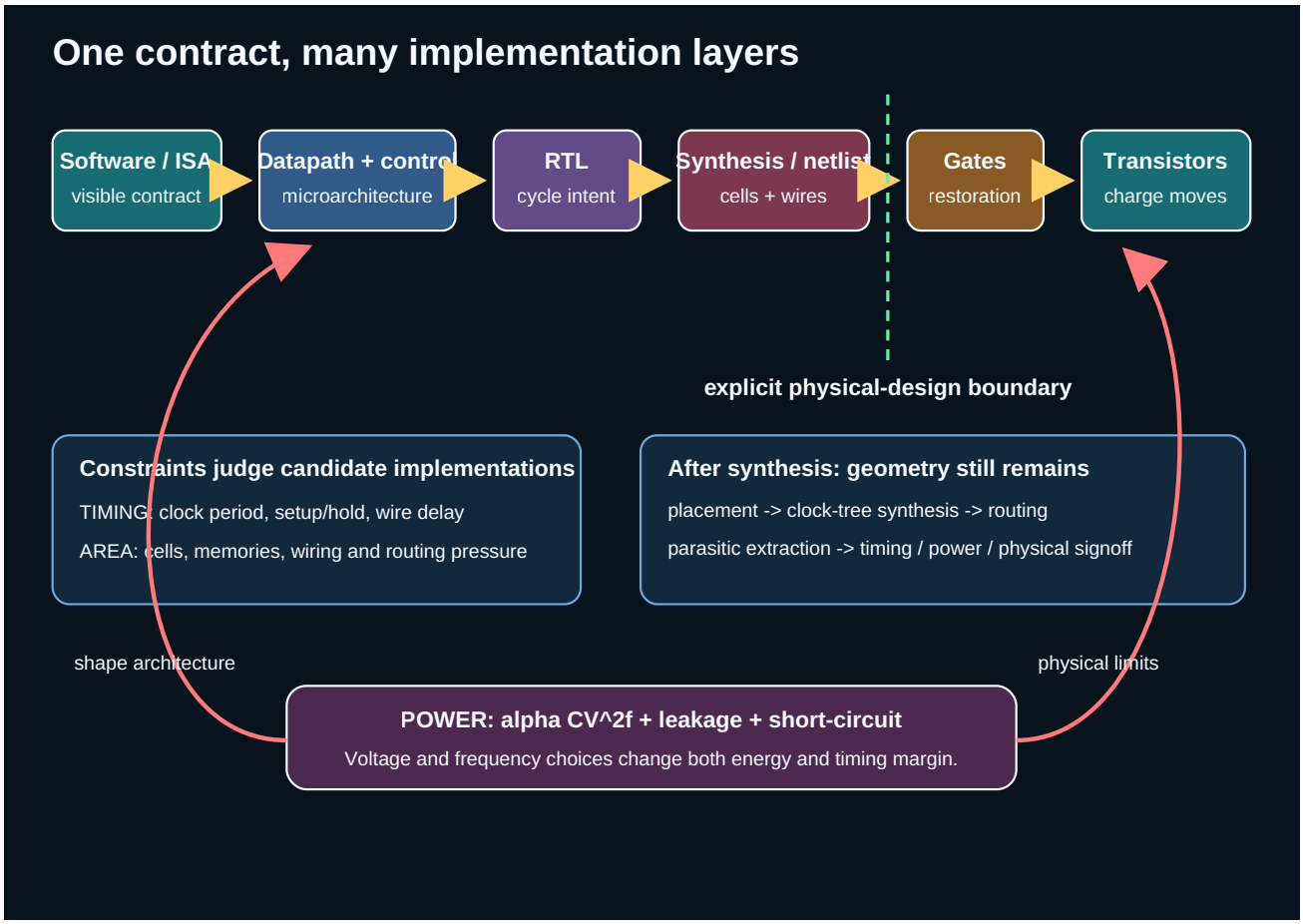
Week 1 introduced physical bits, feedback, registers, a minimal CPU, architectural state, and addressed memory. Week 4 revisits only the mechanisms needed to support those abstractions: transistor switching, restoring logic, bit-cell storage, timing, RTL, and synthesis. It does not replace the Week 1 programmer-visible model with circuit details; it connects the layers.

“Week” names a mastery module, not a calendar deadline. Spend longer when a waveform, timing diagram, or synthesized netlist contradicts a prediction. Reading all seven sections is not completion: preserve evidence across Explain, Draw, Observe, and Build.

Why these seven days belong together

This module follows one causal chain without pretending every abstraction is the same thing:

```
MOSFET controlled conduction
  → complementary pull-up/pull-down networks
  → restoring logic gates
  → delay, capacitance, fan-out, energy, and noise constraints
  → clocked state with setup/hold and clock-to-Q timing
  → SRAM feedback and DRAM charge as array bit-cell mechanisms
  → register/cache/main-memory latency, density, and cost tradeoffs
  → SystemVerilog RTL and simulation
  → elaborated design and generic cells
  → optimized, technology-mapped netlist
  → CPU datapath/control beneath the ISA
  → MMU, privilege, exceptions, and operating systems
```



The physical-design boundary follows synthesis; timing, area, and power feed back into every implementation choice.

The transistor explanation tells us why voltage, charge, delay, and power cannot be ignored. The gate abstraction makes larger reasoning possible. Timed sequential elements create stable state boundaries. RTL describes data movement and operations between those boundaries. Synthesis then chooses and rewrites a netlist that implements that RTL under a target model and constraints. Placement, clock-tree synthesis, routing, and signoff are later steps; synthesis alone does not create physical geometry. We name that boundary concisely so we can reason honestly about CPU timing, area, and power—not to turn this module into an ASIC-design course.

Daily roadmap and exact files

Day 1 — The MOSFET as a Controlled Switch

Build a useful device-level model of NMOS and PMOS behavior: terminals, gate-controlled channels, body-diode caveats, charge, and why a MOSFET is not an ideal Boolean switch.

Lesson:

```
04-cpu-and-chip-design/sections/day-001-mosfet-as-a-switch.md
```

Evidence: explain channel control without saying current enters the insulated gate, identify pull-up and pull-down roles, and relate a changing node voltage to charging or discharging capacitance.

Day 2 — CMOS Inverter and Noise Margins

Derive the CMOS inverter from complementary pull-up and pull-down devices. Explain static behavior, valid voltage ranges, restoration, transition behavior, and noise margins before composing larger gate networks.

Lesson:

```
04-cpu-and-chip-design/sections/day-002-cmos-inverter-and-noise-margins.md
```

Evidence: draw the complementary inverter, explain restoration and noise margins, and distinguish low settled static current from switching current.

Day 3 — CMOS NAND, NOR, and Gate Networks

Derive NAND and NOR from complementary series/parallel transistor networks, then connect those restoring gates to larger Boolean functions.

Lesson:

```
04-cpu-and-chip-design/sections/day-003-cmos-nand-nor-and-gate-networks.md
```

Evidence: draw and trace NAND/NOR pull-up and pull-down networks for every input combination, then explain what their truth tables omit physically.

Day 4 — Delay, Capacitance, Fan-Out, and Power

Connect load capacitance and finite drive to propagation delay; distinguish rise/fall behavior; reason about fan-out, dynamic energy, leakage, short-circuit current, and the voltage/frequency tradeoff.

Lesson:

```
04-cpu-and-chip-design/sections/day-004-delay-capacitance-fanout-and-power.md
```

Evidence: annotate a loaded gate waveform, compare fan-out cases, and use `P_dynamic ≈ α C V² f` without treating it as a complete chip power equation.

Day 5 — Storage Cells, Sequential Timing, and Metastability

Move from feedback to 6T SRAM, DRAM charge storage, latches, and edge-triggered registers. Distinguish a bit-cell circuit from architectural memory; connect register/cache/DRAM density, cost, and latency to cache misses; then use setup, hold, clock-to-Q, skew, jitter, metastability, and clock-domain crossing discipline to explain when state transfer is reliable.

Lesson:

```
04-cpu-and-chip-design/sections/day-005-sequential-timing-and-metastability.md
```

Evidence: explain SRAM read/retention and DRAM sensing/restoration/refresh, draw a register-to-register path, check setup and hold relationships, and explain why synchronizers reduce rather than eliminate metastability risk.

Day 6 — SystemVerilog RTL and Waveforms

Describe a small ALU/register stage with `always_comb`, `always_ff`, nonblocking assignments, complete selection, reset, and valid state. Use a testbench and waveform to distinguish combinational result from captured state.

Lesson:

```
04-cpu-and-chip-design/sections/day-006-systemverilog-rtl-and-waveforms.md
```

Evidence: simulate reset and enable cases, annotate state changes by clock edge, identify `always_ff` as a register abstraction rather than a bit-cell schematic, and explain why synthesizable RTL is only a subset of everything a SystemVerilog simulator can execute.

Day 7 — Synthesis, Netlists, and Physical-Design Overview

Follow RTL through elaboration, generic inference, optimization, and technology mapping. Use one small Yosys observation, then reconnect the netlist to CPU timing/area/power and the software→ISA→datapath→gates path.

Lesson:

```
04-cpu-and-chip-design/sections/day-007-synthesis-netlists-and-physical-design-overview.md
```

Challenges:

```
04-cpu-and-chip-design/challenges/day-007-synthesis-demo.sv
04-cpu-and-chip-design/challenges/day-007-synthesis-testbench.sv
04-cpu-and-chip-design/challenges/day-007-yosys-synthesis-lab.sh
```

Evidence: preserve optional simulation output plus generic/mapped statistics, netlists, and DOT graphs from the single lab; identify inferred mux/adder/flop structures and state the evidence handoff explicitly: ngspice supports analog-model behavior, HDL simulation supports exercised behavior, and synthesis supports structure—not silicon timing or layout.

Adjustable workload guide

The nominal budget is approximately 21 focused hours:

```
Day 1   3 h   MOS device model and switching observations
Day 2   3 h   CMOS inverter, restoration, noise margins
Day 3   3 h   NAND/NOR and complementary gate networks
Day 4   3 h   delay, loading, fan-out, dynamic/static power
Day 5   3 h   SRAM/DRAM cells, registers, timing, metastability, CDC
Day 6   3 h   SystemVerilog RTL, ALU/register, waveforms
Day 7   3 h   one synthesis observation and systems bridge
```

This is a planning estimate, not a completion rule. Split labs across sessions when needed, repeat a waveform or synthesis run when the output is surprising, and shorten familiar exposition only when the evidence remains complete.

Integrated weekly evidence

Explain

Without notes, tell one connected story:

1. a MOS gate voltage controls channel conduction and therefore how a node capacitance can charge or discharge;
2. complementary transistor networks produce restoring logic levels;
3. finite resistance, capacitance, activity, and leakage create delay and power costs;
4. 6T SRAM uses cross-coupled feedback while DRAM uses capacitor charge, destructive sensing/restoration, leakage management, and refresh;
5. a bit-cell circuit is not architectural memory, and density/cost/latency tradeoffs help explain registers, cache SRAM, main DRAM, and cache misses;
6. timed sequential elements divide combinational work into state transitions;
7. SystemVerilog expresses intended combinational and sequential behavior;
8. simulation executes a model over time, while synthesis constructs and transforms an implementation graph;
9. elaboration resolves parameters, hierarchy, widths, and generate choices;
10. generic synthesis infers operations and storage, optimization changes structure while preserving required behavior, and technology mapping selects target-supported cells/resources;
11. constraints define clocks, I/O assumptions, and design limits used to judge candidate implementations;
12. synthesis still has not placed or routed physical resources;
13. timing, area, and power pressure feeds back into CPU choices such as pipeline depth, datapath width, cache organization, and clock frequency;
14. the ISA hides those implementation details while exposing the state and events on which compilers, kernels, and ToyOS depend.

Draw

Produce one diagram containing:

- an NMOS/PMOS CMOS inverter and one two-input CMOS gate;
- a loaded gate with a capacitance and delayed output;
- a 6T SRAM cell at qualitative block level and a 1T1C DRAM cell, clearly labeled “bit cells, not architectural memory”;
- launch register → combinational path → capture register, with setup and hold windows;
- synthesizable RTL for a selected add feeding a register;
- RTL → elaborated hierarchy → generic mux/adder/flop → optimized graph → mapped cells/resources;
- software operation → ISA instruction → control/datapath → gates, with a concise “physical implementation remains” boundary.

Label where voltage behavior, Boolean behavior, cycle behavior, timing constraints, and physical geometry become relevant. Do not draw synthesis as if it emits a finished chip.

Observe

Keep a compact evidence bundle:

- one device/gate switching observation or carefully annotated calculation;
- gate waveforms showing finite transition delay;
- one setup/hold timing exercise and metastability explanation;
- combinational and sequential RTL simulation logs/waveforms;
- Yosys version and commands;
- generic and mapped `stat` output from the single Day 7 observation;
- its generated generic/mapped netlists and DOT graphs;
- a bounded handoff: ngspice demonstrates analog behavior of its model, HDL simulation demonstrates exercised model behavior, and synthesis exposes structure; none establishes final silicon timing or layout;
- a short note explaining why cell counts can change with optimization and target mapping.

Build

Complete or extend the labs so one datapath:

1. selects one of two input operands;
2. adds a constant or second operand;
3. captures the result in a clocked register with reset and enable;
4. passes an RTL simulation test;
5. synthesizes to observable mux, arithmetic/logic, and flip-flop structure;
6. can be modified and re-synthesized once to compare structure without claiming cell counts are final silicon area or transistor count.

Retain the loop:

```
predict → simulate → synthesize → inspect → explain → change → compare
```

Integrated mastery checkpoint

Trace one software-visible value through the entire stack:

1. How does a C operation become one or more ISA instructions?
2. How do fetch/decode and control select register and ALU actions?
3. How does RTL describe that combinational work and captured state?
4. Which mux, adder, and flip-flop structures can synthesis infer?
5. How do restoring gates compose those structures?
6. How does transistor conduction move charge to change a represented bit?
7. How do SRAM feedback and DRAM capacitor charge retain array bits, and why is neither cell by itself architectural memory?
8. How can a cache miss turn an architectural load into a lower-level DRAM access and line fill?
9. Why do fan-out, setup/hold, and propagation delay constrain the clock?
10. Why do area and power constrain datapath, pipeline, and cache choices?

11. What does the ISA expose while hiding those implementation choices?
12. Why do MMU, privilege, exception, and interrupt mechanisms form the next bridge from CPU hardware back to kernels and ToyOS?

Mastery is demonstrated when every arrow names both the representation and the responsible tool or physical mechanism:

```
software → ISA → datapath/control → RTL → netlist
         → gates → transistor switching
         → timing/area/power constraints on the CPU design
         → MMU/privilege/exceptions → operating system
```

End-of-module reflection

Record in your own words:

- strongest understanding;
- remaining confusion;
- one prediction contradicted by simulation or synthesis;
- one place where a behavioral model was mistaken for physical reality;
- connection to CPU, kernel, driver, FPGA, or virtualization work;
- next revision action;
- whether to advance toward CPU microarchitecture, MMU/privilege, and ToyOS or repeat one lab.

Do not auto-fill confidence, completion, hours, or mastery. Those remain the learner's evidence-based assessment.

Day 1 — The MOSFET as a Controlled Switch

Target time: approximately 2–3 hours

- Historical and device model: 65–75 minutes
- Predictions, sketches, and worked reasoning: 30–40 minutes
- ngspice experiment and evidence: 55–70 minutes
- Explain/Draw/Observe/Build checkpoint: 20–30 minutes

> A CPU appears to manipulate symbols. At its lowest active layer, however, changing a bit means moving charge through devices whose conductance depends on voltage. What kind of device can make that reliable enough to repeat billions of times?

Why this day exists

Earlier lessons treated a transistor as an ideal controlled switch. That abstraction was useful: it let us derive Boolean functions without stopping for device physics. It also hid the questions that determine whether a gate works in silicon:

- What does the control voltage actually change?
- Is a MOSFET ever perfectly open or perfectly closed?
- Why is there a threshold rather than one exact switching voltage?
- Why does a conducting transistor still have a voltage drop and finite current?
- Where does the output voltage come from?
- Why does changing an output take time?

Today we replace the ideal switch with a small, useful electrical model. The goal is not semiconductor-device specialization and not quantum mechanics. The goal is enough physical grounding to reason honestly about CMOS gates, delay, power, and restored logic.

The path is:

```
voltage on a gate terminal
    → electric field changes channel conductivity
    → current moves charge
    → node voltage changes
    → another device senses that voltage
    → a switching network realizes logic
```

The last arrow belongs mainly to Day 2. Today earns the first four.

---

1. The engineering problem before MOS logic

Relays provided controllable conducting paths, but their moving contacts were large, slow, noisy, and subject to wear. Vacuum tubes removed mechanical motion and enabled much faster electronic switching, but occupied substantial space and consumed considerable power. Bipolar transistors made switching solid-state, smaller, and more reliable.

The metal-oxide-semiconductor field-effect transistor, or **MOSFET**, offered another decisive property: its control terminal is insulated from the conducting channel by a dielectric. Ideally, steady gate current is nearly zero. A gate is controlled primarily by voltage and electric field rather than by a continuous input current.

MOS devices also fit planar fabrication and scaling extremely well. When complementary n-channel and p-channel devices are paired as CMOS, stable logic states can ideally avoid a direct DC path from the supply to ground. That combination—density, manufacturability, and low static power—made CMOS the dominant foundation of large digital integrated circuits.

This history is not a story in which each technology instantly replaced the last. Relays, tubes, bipolar devices, and MOS devices overlap in time and remain useful in different applications. The important engineering sequence is that each technology changed the limits on speed, size, reliability, drive, and power.

If you are curious about the wider family, use the optional Transistor Family Primer. It briefly compares BJTs, JFETs, enhancement/depletion MOSFETs, planar MOSFETs, FinFETs, GAAFETs, and power-device relatives, then returns to why CMOS is the relevant path for CPU logic.

---

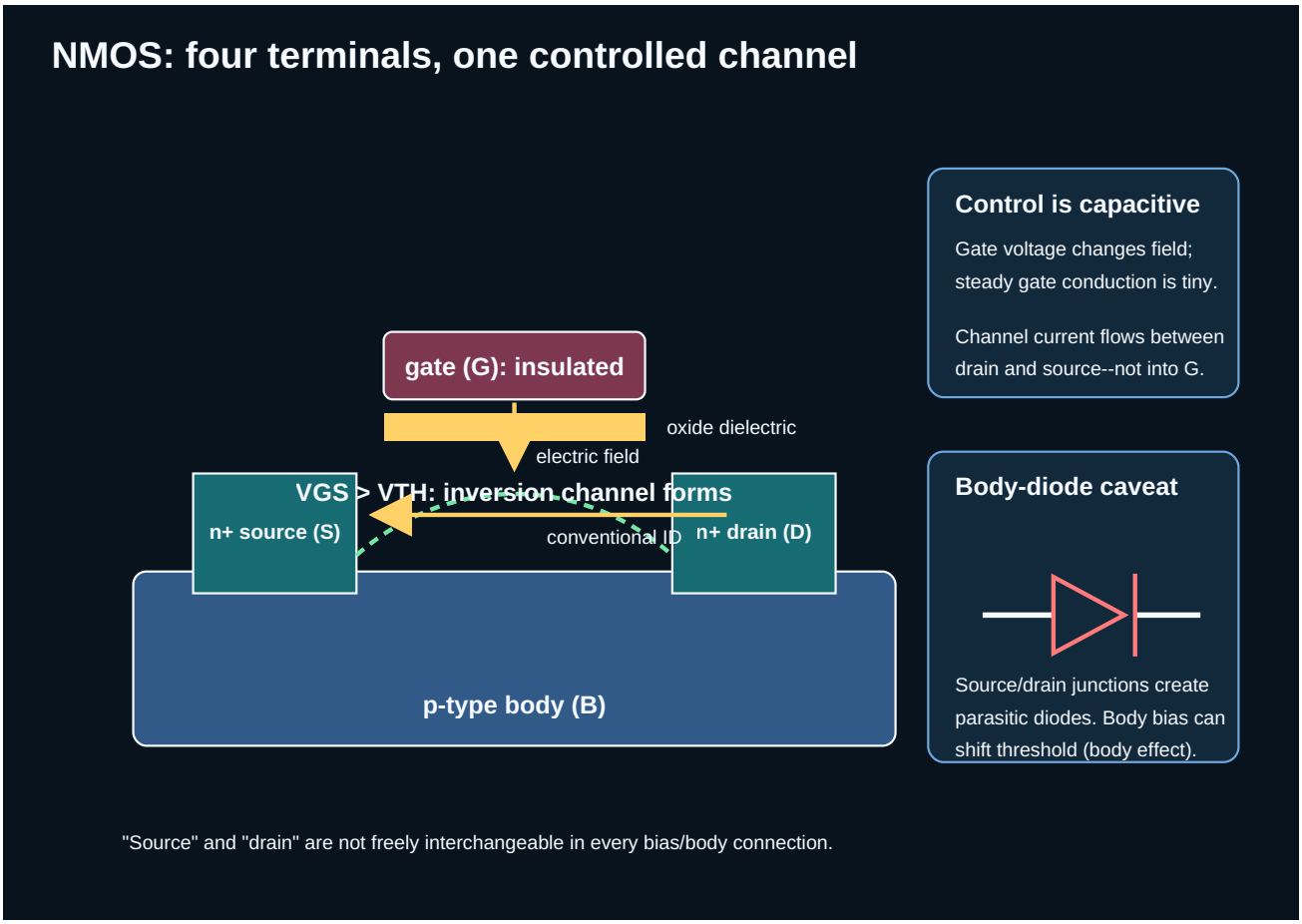
2. Four terminals, three voltages, one useful abstraction

An n-channel MOSFET has four physical terminals:

```
                  drain D
                     |
gate G ── electric field controls channel
                     |
                  source S

body or bulk B influences the channel too
```

Digital schematics often tie the nMOS body to the lowest supply and the pMOS body to the highest supply, then visually suppress the body connection. Do not forget that it exists. Body voltage can shift threshold through the **body effect**, and source/drain junctions create parasitic diodes.



The insulated gate controls channel conductivity; the body terminal and parasitic junction diodes still matter.

For a first nMOS model, use:

```
VGS = VG - VS      gate-to-source voltage
VDS = VD - VS      drain-to-source voltage
VTH                threshold parameter
```

The gate dielectric blocks an ordinary conductive path from gate to channel, but the gate, oxide, and semiconductor form a capacitor. Applying `VGS` changes the electric field and therefore the charge distribution beneath the gate.

For an enhancement-mode nMOS:

- low `VGS`: no strong inversion channel; drain current is very small;
- `VGS` above threshold: a conducting channel forms;
- larger gate overdrive `VGS - VTH`: the channel generally conducts more strongly.

This is the switch intuition:

```
gate LOW   → nMOS weak/off → path approximately open
gate HIGH  → nMOS on       → path approximately conducting
```

It is an approximation, not a new physical mode that erases analog behavior.

A necessary vocabulary correction

Engineers often say “current flows from drain to source” for conventional current in an nMOS. Electrons, which carry the channel current, drift in the opposite direction. Circuit analysis normally uses conventional current. Choose one convention and label it rather than silently changing directions.

---

3. Threshold is not the gate's logic threshold

The MOSFET threshold voltage `VTH` is a device-model quantity associated with the onset of strong inversion. It is **not** all of the following:

- a perfect boundary below which current is exactly zero;
- the input voltage at which every inverter changes logical state;
- a universal constant for all transistors;
- necessarily half the supply voltage;
- a guarantee that the transistor is a low-resistance switch.

Below threshold, real MOSFETs have **subthreshold current**. Above threshold, drive strength grows continuously. Threshold varies with fabrication process, temperature, body bias, device geometry, and the definition/measurement method. Small devices also exhibit effects omitted by the long-channel model.

Later, the CMOS inverter's switching point will emerge from competition between its nMOS and pMOS currents. That circuit-level point depends on both devices and their sizing, not merely on one device's `VTH`.

Keep three ideas separate:

```
device threshold      model of channel formation
logic input limits    voltages guaranteed to be interpreted safely
inverter trip region  where pull-up and pull-down compete
```

Collapsing these into one “threshold” causes persistent confusion.

---

4. The smallest device model a systems programmer needs

We need a behavioral map, not a device-design derivation:

Condition	Useful first approximation	Important caveat
gate well below threshold	channel is weak; switch looks open	real devices leak
gate above threshold	channel conductance rises	there is no one fixed “on resistance”
conducting with small drain-source voltage	behaves roughly resistively	resistance depends on bias and geometry
conducting while drain-source voltage is large	current is limited by device behavior	“saturation” does not mean an ideal closed switch

The labels **cutoff**, **triode/linear**, and **saturation** name operating regions of a simplified MOS model. A transistor can move through several of them during one gate transition. For this course, their value is explanatory: they tell us why a MOSFET is not a mechanical contact and why delay/current change throughout a transition.

Modern transistors need much richer models for design. We intentionally stop before short-channel equations, fabrication details, and process-specific parameters. The lab's simple Level 1 model exposes the shape of behavior; it does not predict a commercial CPU.

---

5. Predict the transistor curves

Imagine holding `VDS` at three values while sweeping `VGS` from 0 V to 3.3 V.

Before simulation, sketch current on the vertical axis and `VGS` on the horizontal axis.

1. Below `VTH`, what does the simple model predict?
2. Which fixed `VDS` should permit the largest current?
3. At high `VGS`, could the smallest `VDS` case enter the linear region?
4. Should current jump to one fixed “on” value at threshold?

Now imagine fixing `VGS` high and increasing `VDS`:

```
small VDS                    larger VDS
resistive/triode  ──────────→  saturation region
```

The curve bends because the channel is not a mechanical contact with one resistance. The systems-level observation is enough: changing a control voltage continuously changes available drive current, and that current sets how quickly downstream capacitance can move to a new logic level.

---

6. An nMOS used as a pull-down switch

Connect a resistor to the supply and an nMOS to ground:

```
            VDD
             │
          resistor
             │
             ├──── VOUT
             │
           nMOS
             │
           GND

VIN controls the nMOS gate
```

When `VIN` is low, the nMOS is weak. The resistor pulls `VOUT` toward `VDD`. When `VIN` is high, the nMOS conducts and removes charge from the output node, pulling it toward ground.

This circuit inverts, but it is not modern complementary CMOS:

- in the LOW-output state, current continuously runs through the resistor and nMOS;

- the resistor is a weak pull-up and can make rising transitions slow;
- output LOW depends on a resistance ratio and is not exactly ground;
- a practical integrated resistor can consume substantial area.

It remains an excellent bridge from one transistor to a restoring gate.

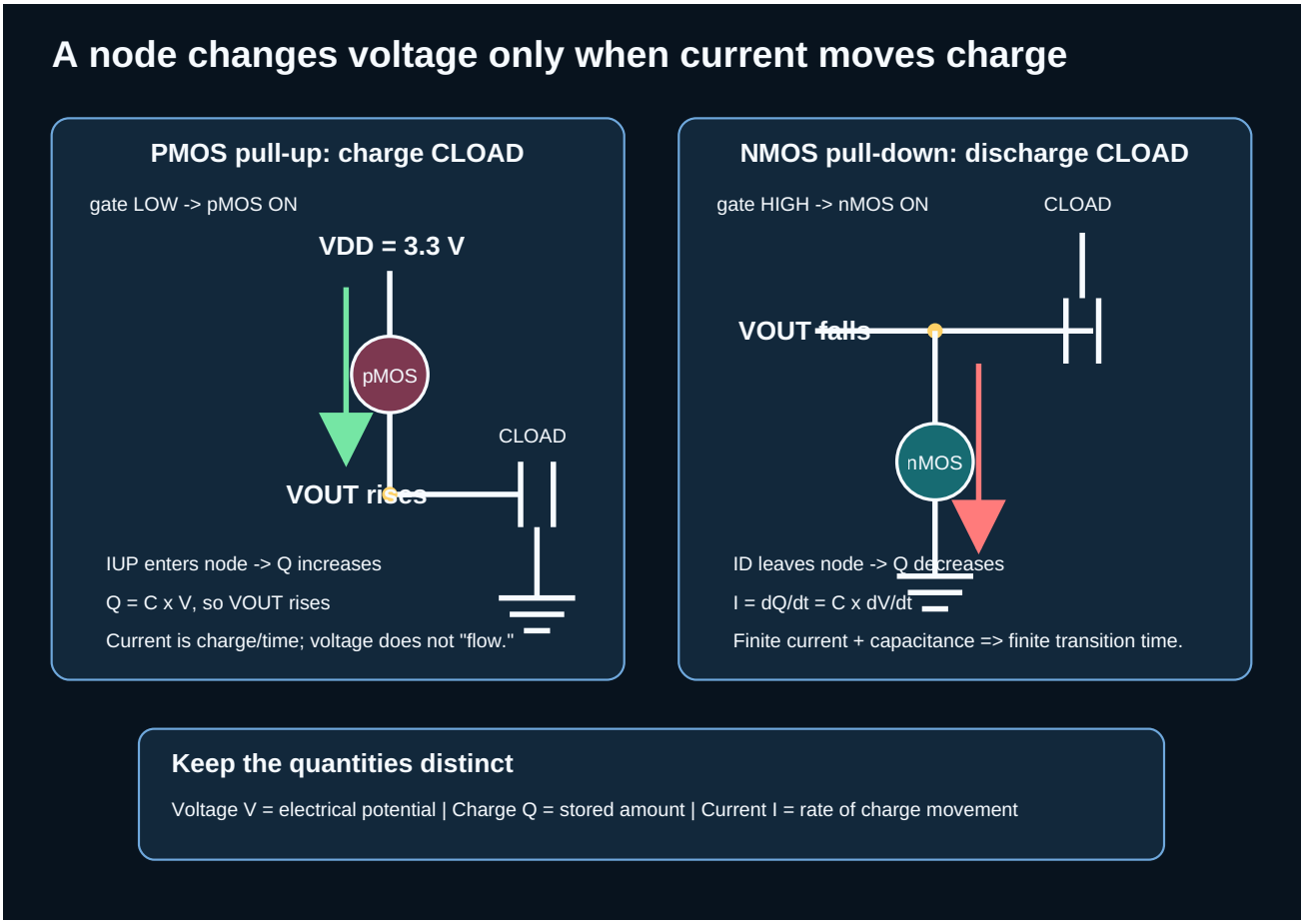
Voltage does not “flow”

When the nMOS turns on, voltage does not pour out of the node. Current moves charge. The output node has capacitance—intentional and parasitic. Its voltage is related to stored charge:

```
Q = C V
I = dQ/dt = C dV/dt
```

Therefore:

```
dV/dt = I/C
```



Current moves charge into or out of capacitance; the resulting charge change produces the node-voltage change.

A finite current needs finite time to change voltage across capacitance. This is the physical seed of propagation delay.

First-order RC intuition

When the transistor is off, the output charges through the resistor. If the load is approximated as a capacitor:

```
VOUT(t) ≈ VDD(1 - e^(-t/RC))
```

The time constant is `τ = RC`. After one time constant the node has moved about 63% of the way toward its final value. It never waits for a symbolic “bit assignment”; its analog voltage changes continuously.

When the nMOS turns on, its nonlinear channel provides a discharge path. An `RON × C` estimate is useful but approximate because `RON` changes as `VGS` and `VDS` change.

---

7. pMOS: complementary polarity, not “backward nMOS”

A p-channel MOSFET is naturally described relative to its source, usually connected toward the high rail:

```
VSG = VS - VG
VSD = VS - VD
```

It turns on strongly when `VSG` exceeds the magnitude of its threshold:

```
gate LOW relative to source  → pMOS on
gate HIGH near source        → pMOS off
```

This opposite control polarity makes pMOS useful as a pull-up:

```
VDD ── pMOS ── output
```

Do not infer that source and drain labels can always be swapped without consequence. MOS structures can be geometrically similar, but body connections, parasitic diodes, bias conditions, and model conventions matter.

Electron mobility is generally higher than hole mobility, so for the same geometry an nMOS is usually stronger than a pMOS. CMOS libraries often make the pMOS wider to balance rising and falling behavior. “Wider” also increases capacitance, area, and dynamic energy; sizing is a trade-off.

---

8. The gate terminal draws no DC current—almost

The ideal insulated gate suggests zero input current. For digital reasoning, the better sentence is:

> A MOS gate ideally draws negligible steady-state conductive current, but it has capacitance and real devices leak.

Changing gate voltage requires charging or discharging gate capacitance. Thus an upstream gate must deliver transient current. At rest, leakage can come from subthreshold conduction, gate tunneling, reverse-biased junctions, and other mechanisms. Leakage becomes especially important at high transistor counts and small geometries.

This distinction will matter when discussing power:

```
steady ideal gate input current       approximately zero
current while changing gate voltage   definitely not zero
real static leakage                   not zero
```

---

9. Lab — see the curve and then use the device

The lab is:

`04-cpu-and-chip-design/challenges/day-001-mosfet-switch-lab.cir`

It performs two analyses:

1. a DC sweep of gate voltage for nMOS devices held at several drain voltages;
2. a transient simulation of a resistor-loaded nMOS inverter.

It uses a deliberately simple Level 1 model. No external model files, GUI, or network access are required.

Predict

Write down:

1. At what approximate gate voltage should drain current become noticeable in this model?
2. Rank the three fixed-`VDS` current curves near `VGS = 1.0 V`.
3. When the pulse input rises, should `VOUT` rise or fall?
4. Which output transition should the resistor make relatively slow?
5. Will output LOW be exactly 0 V?

Simulate

From the repository root:

```
mkdir -p /tmp/mosfet-switch-lab
ngspice -b \
  -o /tmp/mosfet-switch-lab/ngspice.log \
  04-cpu-and-chip-design/challenges/day-001-mosfet-switch-lab.cir
```

The netlist writes its data into `/tmp/mosfet-switch-lab/`:

```
nmos-transfer.dat      text columns: VGS and drain-current curves
nmos-switch-transient.dat
ngspice.log            analysis messages and measured values
```

The `.dat` files are whitespace-separated numeric text suitable for a spreadsheet, Python, gnuplot, or careful inspection with `less`. This avoids requiring a graphical waveform viewer.

Inspect the first rows:

```
less /tmp/mosfet-switch-lab/nmos-transfer.dat
less /tmp/mosfet-switch-lab/nmos-switch-transient.dat
cat /tmp/mosfet-switch-lab/ngspice.log
```

Optional plot with gnuplot, if installed:

```
gnuplot -persist <<'PLOT'
set key left top
set xlabel "VGS (V)"
set ylabel "ID (A)"
plot "/tmp/mosfet-switch-lab/nmos-transfer.dat" \
  using 2:3 with lines title "VDS=0.05 V", \
  using 2:4 with lines title "VDS=0.50 V", \
  using 2:5 with lines title "VDS=3.30 V"
PLOT
```

With `wr_singlescale`, ngspice writes one analysis scale followed by the requested vectors. The transfer columns are the DC sweep scale, `VGS`, and the three drain currents. The header names them explicitly.

Observe

Find evidence for each statement:

- current varies continuously rather than switching between only zero and one fixed value;
- the simple model's below-threshold current is effectively zero apart from ngspice's tiny numerical conductances because the model omits realistic subthreshold leakage;
- low `VDS` eventually limits current differently from high `VDS`;
- the transistor discharges output capacitance when on;
- the resistor restores the output HIGH when the transistor turns off;
- the rise and fall are not instantaneous or necessarily symmetric.

Use the measured rise and fall times in the log, but inspect the waveform data too. A scalar measurement can conceal an unexpected waveform.

Explain

Complete these sentences in your own words:

1. Raising `VGS` changes drain current because …
2. `VTH` is not a perfect digital boundary because …
3. `VOUT` changes only when charge moves because …
4. The output returns HIGH after nMOS turn-off because …
5. This resistor-loaded inverter wastes static power when LOW because …

---

10. Read the netlist as an engineering model

The most important lines are conceptually:

```
.model NDEV NMOS (LEVEL=1 VTO=0.70 KP=200u ...)
Mcurve drain gate 0 0 NDEV W=10u L=1u
```

SPICE MOSFET terminals are ordered:

```
Mname drain gate source body model parameters
```

Getting source/body order wrong can produce a simulation that runs but models the wrong circuit.

The fixed drain-voltage sources let ngspice measure current at selected `VDS` values. The resistor-loaded output includes a capacitor:

```
VDD → resistor → VOUT → capacitor and nMOS → ground
```

The capacitor does not mean output capacitance comes from one explicit component in a chip. It stands in for drain junction capacitance, wiring, downstream gate capacitance, and any intentional load.

Model versus measurement

The simulation can establish what this netlist and model predict. It cannot establish:

- the characteristics of a particular manufactured transistor;
- safe voltages for arbitrary hardware;
- a modern process's leakage or short-channel behavior;
- exact CPU gate delay.

Simulation is an executable consequence of assumptions. Always name the assumptions.

---

11. Bridge quickly back to CPU and software

A processor does not expose `VGS` or drain current to normal software. Designers compose transistors into characterized gates, gates into datapaths and storage, and those blocks into an implementation of an ISA. Yet every architectural state change still rests on the mechanism from this lesson:

```
software instruction
  → decoded control signals
  → selected transistor networks conduct
  → charge moves on internal nodes
  → voltages settle into valid ranges
  → a clocked element captures new state
```

For example, an integer add can switch operand-selection logic, many adder stages, result-routing logic, and register-write controls. The transistors do not understand addition. Their local currents collectively make the voltage pattern that the architecture calls a result.

This explains three system facts we will revisit:

- switching more capacitance costs more energy;
- finite current and capacitance limit clock speed;
- violating voltage, temperature, or timing assumptions can break the architectural abstraction and appear to software as incorrect execution.

We now leave single-device depth and move to restoring gates.

---

12. Explain, Draw, Observe, Build

Explain

Without notes:

1. Explain how gate voltage controls an nMOS channel.
2. Distinguish cutoff, triode, and saturation in the teaching model.
3. Explain why a MOSFET is not an ideal switch.
4. Explain why the insulated gate still causes dynamic current upstream.
5. State pMOS turn-on conditions using source-relative voltage.
6. Explain why output voltage changes require movement of charge.

Draw

Draw and label:

1. an nMOS with drain, gate, source, and body;
2. `VGS`, `VDS`, and conventional drain current;
3. qualitative `ID` versus `VGS` curves at three `VDS` values;
4. the resistor-loaded nMOS inverter with output capacitance;
5. charge entering the output node on a LOW-to-HIGH output transition and leaving on a HIGH-to-LOW transition.

Observe

Submit:

- the exact ngspice command and version;
- both generated data files and the log;
- your five predictions;
- two correct predictions and one surprise;
- one limitation caused by the Level 1 model.

Build

Make a copy in `/tmp`, leaving the course file unchanged:

```
cp 04-cpu-and-chip-design/challenges/day-001-mosfet-switch-lab.cir \
  /tmp/mosfet-switch-lab/experiment.cir
```

Choose one modification:

- double the output capacitance;
- double the pull-up resistance;
- halve nMOS width;
- change `VTO` while clearly recording that this changes the model.

Predict the direction of rise time, fall time, and steady output levels. Edit the `/tmp` copy, rerun it, and explain the result. Do not search for one magic “correct” delay; connect the changed parameter to current, charge, or resistance.

13. Mastery checkpoint

Close the lesson and answer:

> How can a voltage at one terminal control a different current path, and how can that current path turn an analog node voltage into the physical basis of a bit?

A strong answer includes:

- electric-field control through an insulated gate;
- channel formation as a continuous device behavior;
- finite on conductance and nonzero off leakage in real devices;
- current moving charge into or out of node capacitance;
- voltage as a consequence of charge and capacitance;
- external pull-up or pull-down networks setting the final level;
- thresholds imposed by the next stage, not by nature declaring a bit.

If the answer says only “HIGH means on,” return to the transistor curves.

Mental model at the end

```
The gate does not send a Boolean command into the transistor.
A voltage creates an electric field.
The field changes channel charge and conductivity.
Conductivity permits current.
Current changes stored charge on circuit nodes.
Stored charge determines voltage.
Later stages interpret voltage ranges as logic.
```

The MOSFET is valuable precisely because this analog chain can be arranged so that digital abstractions become robust.

Why? notebook

1. Why is a MOSFET gate voltage-controlled even though changing it requires current?
2. Why does `VTH` not define a complete logic interface?
3. Why can an “on” nMOS be in either saturation or triode operation?
4. Why does a larger output capacitance increase transition time?
5. Why is an output node not automatically HIGH merely because its pull-down turned off?
6. Why does a resistor-loaded inverter consume static power in one state?
7. Why is pMOS described with `VSG` more naturally than a positive `VGS`?
8. Why can a simple SPICE model be useful even when it is not process accurate?
9. Why does a wider transistor usually drive more strongly, and what costs accompany width?
10. Where does the energy stored on an output capacitor go when an nMOS pulls the node LOW?

References used selectively

Device and circuit foundations:

- R. Jacob Baker, *CMOS: Circuit Design, Layout, and Simulation*, 4th ed., Wiley-IEEE Press, chapters 6–9. Used for MOSFET operating-region and inverter-level foundations.
- Neil H. E. Weste and David Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., chapters 2–3. Used for the switch, capacitance, delay, and CMOS-design mental models.
- Behzad Razavi, *Design of Analog CMOS Integrated Circuits*, 2nd ed., chapter 2. Useful for a careful introduction to MOS device operation without requiring a fabrication course.

Primary/current tool references:

- ngspice project documentation and current manual, including batch mode, MOS Level 1 models, `dc`, `tran`, `meas`, and `wrdata`: https://ngspice.sourceforge.io/docs.html
- ngspice MOSFET model overview: https://ngspice.sourceforge.io/docs/ngspice-html-manual/manual.xhtml
- Nobel Prize scientific background on the transistor and integrated-circuit engineering lineage: https://www.nobelprize.org/prizes/physics/1956/summary/

Historical context:

- Computer History Museum, “The Silicon Engine,” for the development from transistor invention through MOS and integrated-circuit scaling: https://www.computerhistory.org/siliconengine/

The lab's generic Level 1 parameters are educational and are not taken from a commercial process design kit.

Next bridge: pair an nMOS pull-down with a pMOS pull-up so each stable state is actively restored, then measure where the resulting inverter treats an input as safely LOW, uncertain, or safely HIGH.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Mosfet Switch Lab

Format: ngspice circuit lab

What you will do: Day 1: MOSFET curves and resistor-loaded switch intuition

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
mkdir -p /tmp/day-001-mosfet-switch-lab && (cd /tmp/day-001-mosfet-switch-lab && ngspice -b
"$OLDPWD/04-cpu-and-chip-design/challenges/day-001-mosfet-switch-lab.cir")
```

- View lab source on GitHub

Day 2 — CMOS Inverter and Noise Margins

Target time: approximately 2–3 hours

- Reliable-gate model and CPU bridge: 70–80 minutes
- Prediction and drawing exercises: 25–35 minutes
- Optional ngspice observation: 45–60 minutes
- Explain/Draw/Observe/Build checkpoint: 20–30 minutes

> Software eventually changes voltages. Why do those imperfect analog voltages keep behaving like clean bits after passing through millions of gates?

Why this day exists

Day 1 replaced the ideal switch with a useful systems-level MOSFET model: gate voltage changes channel conductivity; current moves charge; charge changes node voltage. One nMOS plus a resistor could invert a signal, but it had an asymmetric pull-up and wasted steady power in one state.

The CMOS inverter solves the next problem by pairing complementary devices:

```
input LOW  → pMOS pulls output toward VDD
input HIGH → nMOS pulls output toward GND
```

That sentence is only the settled-state story. A reliable gate also needs:

- a transition region between LOW and HIGH;
- enough gain to restore degraded inputs;
- voltage ranges that tolerate noise;
- enough current to charge and discharge its load on time;
- acceptable static and switching energy.

These are not side concerns for analog specialists. They are why a CPU can treat physical voltages as bits, why clock frequency and fan-out are limited, why power rises with activity, and why lower supply voltages make reliable design harder.

The path back to software is direct:

```
instruction changes control/data state
    → selected gate inputs change
    → MOS networks source or sink current
    → internal capacitances gain or lose charge
    → voltages cross receiving-gate limits
    → later pipeline/register state changes
```

We need enough circuit behavior to make that path real. We do not need to design a fabrication process or a production standard-cell library.

1. Build the complementary inverter

Connect a pMOS between the supply and output, and an nMOS between output and ground. Tie both gates to the input:



The shared output has two controlled paths:

- the **pull-up network** can add charge and drive `VOUT` toward `VDD`;
- the **pull-down network** can remove charge and drive `VOUT` toward ground.

Input LOW

With `VIN` near ground:

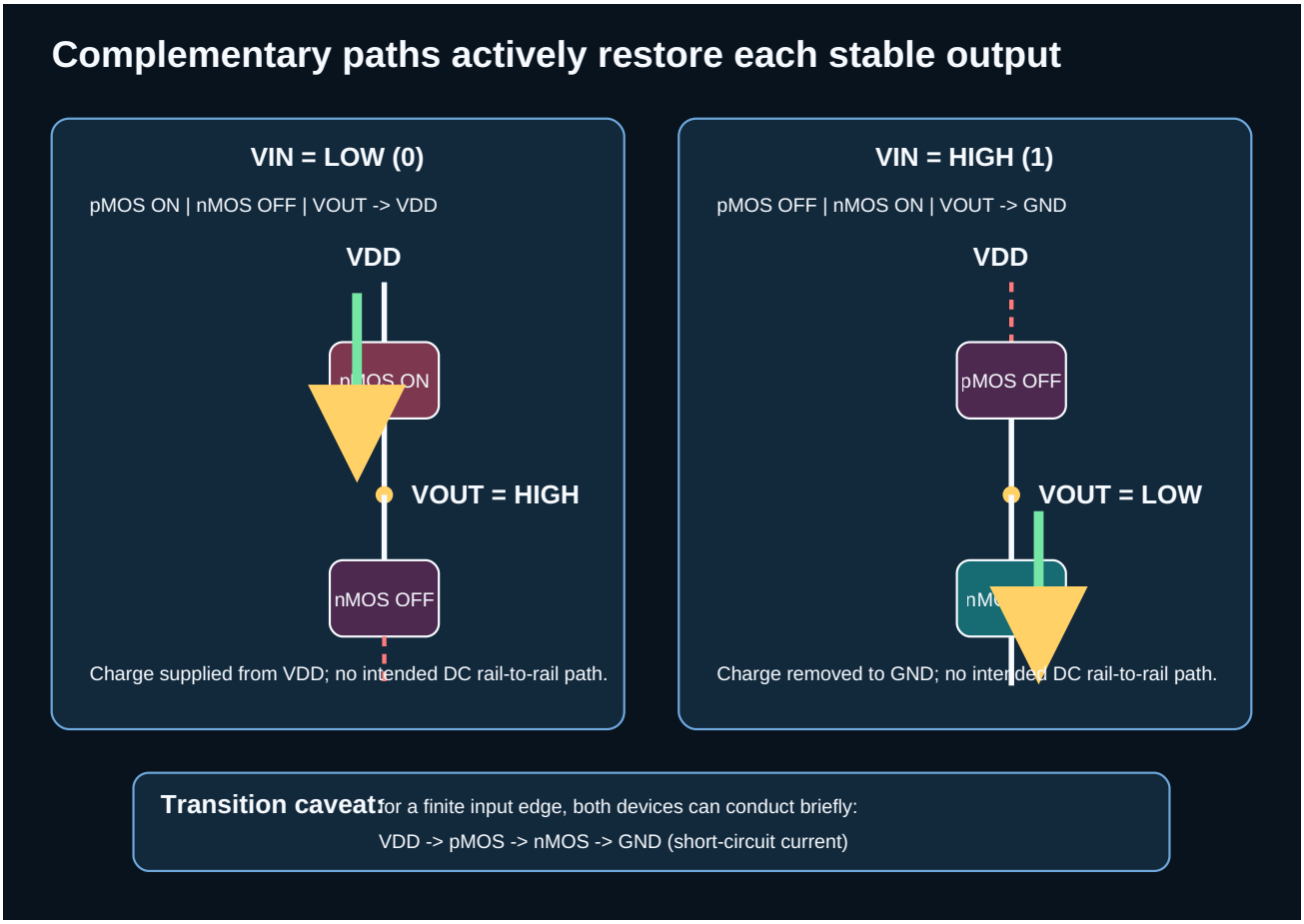
- pMOS is on because its gate is low relative to its source at `VDD`;
- nMOS is off or weak;
- the output charges toward `VDD`;
- the next gate receives a strong HIGH.

Input HIGH

With `VIN` near `VDD`:

- pMOS is off or weak;
- nMOS is on;
- the output discharges toward ground;
- the next gate receives a strong LOW.

In either stable state, the ideal model has no conducting path all the way from `VDD` to ground. This is the central static-power advantage of CMOS. Real devices leak, so “no path” means no intended strong DC path.



Complementary devices provide an active pull-up or pull-down in each stable state, with brief overlap current during transitions.

Why this is inversion

The transistor network does not know logical NOT. It responds locally to voltages. We call the behavior inversion because the settled output ranges map to the opposite logical values:

Input range	Dominant path	Output range	Logical relation
near GND	pMOS pull-up	near VDD	LOW becomes HIGH
near VDD	nMOS pull-down	near GND	HIGH becomes LOW

Logic is the abstraction over repeatable electrical behavior.

---

2. The middle is a real operating region

Suppose `VIN` rises slowly from 0 V to `VDD`.

1. Initially, pMOS is strong and nMOS is weak; output is HIGH.
2. As input enters the middle, nMOS strengthens while pMOS has not fully turned off.
3. Both devices conduct for part of the transition.
4. The output falls rapidly through its middle range.
5. Finally, pMOS becomes weak and nMOS holds the output LOW.

There is no instant at which a physical “0” token becomes a “1” token. Input and output are continuous voltages. The steep middle of the inverter's response is what permits a digital interpretation.

Threshold caveat, repeated because it matters

Do not identify the inverter's switching region with one transistor's threshold voltage. The circuit response depends on:

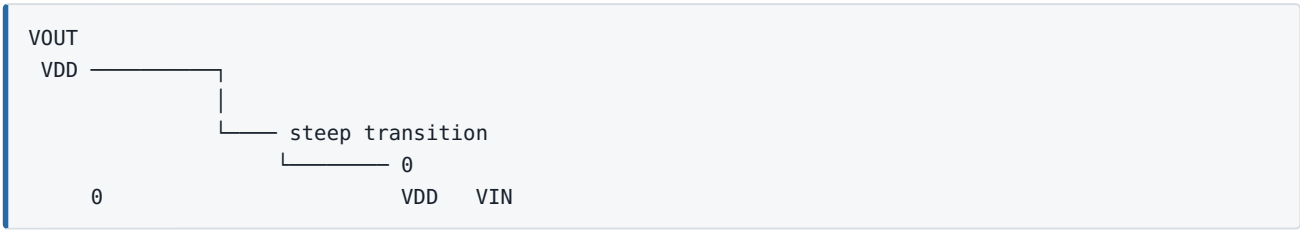
- nMOS and pMOS strengths;
- both device thresholds;
- supply voltage;
- sizing and load;
- process, voltage, and temperature variation.

Production timing and noise guarantees come from characterized libraries across operating corners. Our generic ngspice model illustrates mechanisms; it is not a CPU process model.

---

3. The voltage transfer characteristic

The **voltage transfer characteristic**, or VTC, is the settled output voltage for each slowly varied input voltage:



The inverter is most useful when:

- low inputs produce outputs close to `VDD`;
- high inputs produce outputs close to ground;
- the transition is steep;
- the next gate can classify the output safely.

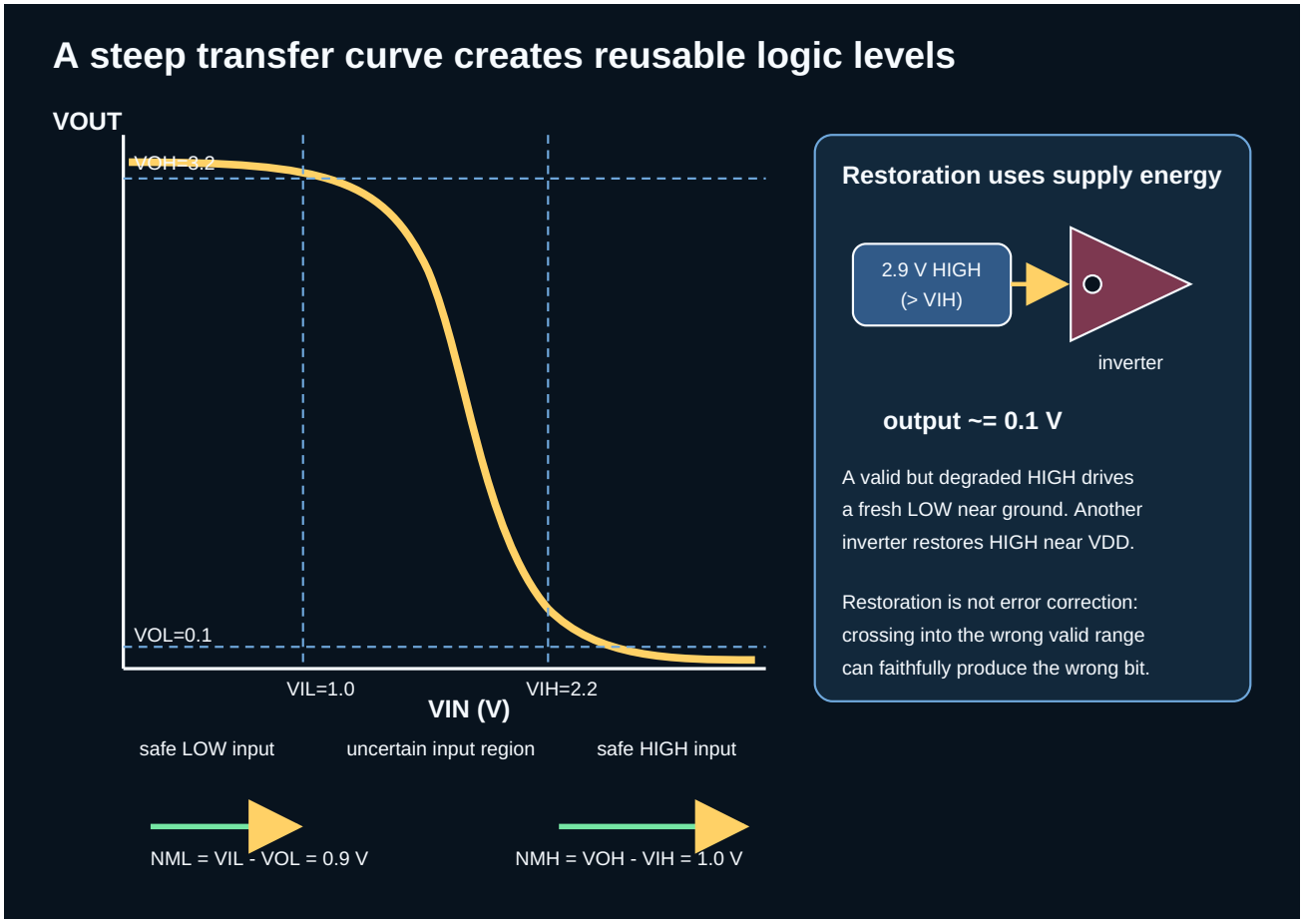
The VTC is a DC view. It does not show propagation time, glitches, or the energy required to move charge. Those need transient reasoning.

Predict the sweep

Before the optional lab, sketch `VOUT` against `VIN` and mark:

- `VOH`: guaranteed or characterized output HIGH level;
- `VOL`: guaranteed or characterized output LOW level;
- `VIL`: largest input guaranteed to be accepted as LOW;
- `VIH`: smallest input guaranteed to be accepted as HIGH;
- the uncertain input interval between `VIL` and `VIH`.

In textbook analysis, `VIL` and `VIH` are often associated with points where the VTC slope is `-1`. That is a useful way to extract margins from a smooth curve. Datasheet or standard-cell limits, however, are specifications backed by characterization; do not substitute a plot from a generic model for a manufacturer guarantee.



The worked VTC separates valid input ranges from the transition region and shows how margin enables restoration.

---

4. Noise margins: room for imperfection

A sending gate promises output levels. A receiving gate promises input interpretation limits. The gaps are the **noise margins**:

```
NML = VIL - VOL
NMH = VOH - VIH
```

Example—not a universal CMOS specification:

```
VDD = 3.3 V
VOL = 0.1 V
VIL = 1.0 V       → NML = 0.9 V
VIH = 2.2 V
VOH = 3.2 V       → NMH = 1.0 V
```

If a LOW output of at most 0.1 V picks up 0.4 V of disturbance, the receiving input is still only 0.5 V, safely below `VIL`. The margin is not permission to ignore signal integrity; it is a compatibility budget between guaranteed ranges.

What “noise” can mean in a system

Noise is not only random fuzz from the environment. A digital path can be disturbed by:

- voltage drop in power and ground distribution;
- coupling from nearby switching wires;
- simultaneous switching current;
- package and board effects;
- slow edges that spend too long in the uncertain region;
- process, voltage, and temperature variation;
- a driver overloaded by excessive fan-out or wire capacitance.

At the software level these effects are usually hidden by hardware margins and timing constraints. When they are not hidden, symptoms can look like rare bit errors, timing failures, machine-check events, or total instability. Software observes the consequence, not a friendly message saying “noise margin exceeded.”

---

5. Restoration: how analog voltages become reusable logic

Suppose an upstream gate's HIGH arrives at 2.9 V instead of an ideal 3.3 V. If 2.9 V is safely above the receiving inverter's `VIH`, the inverter turns its nMOS on strongly and drives its output close to ground. A following inverter can then drive a fresh output close to `VDD`.

```
degraded but valid HIGH
        ↓ receiving inverter
strong LOW near ground
        ↓ another inverter
strong HIGH near supply
```

This is **logic-level restoration**. The gate does not amplify the symbolic meaning of a bit. Its transistor network uses energy from the power supply to produce a new output near a rail.

Three conditions are hidden inside “valid input”:

1. **Voltage:** the input lies inside a guaranteed range.
2. **Time:** it becomes valid early enough and remains valid long enough for the receiving sequential element.
3. **Drive/load:** the source can charge or discharge the path's capacitance quickly enough.

Noise margins address the first. Static timing analysis addresses much of the second and third. Later lessons will connect delay to setup/hold constraints and clocked state.

Restoration is not error correction

An inverter can restore an electrically degraded but correctly classified level. If a disturbance pushes an input across the wrong guaranteed region for long enough, the gate may faithfully produce the wrong logic value. Restoration does not recover lost intent.

---

6. Static power, dynamic power, and the transition interval

Ideal stable-state picture

For a settled LOW or HIGH input, one transistor is off. The simple CMOS model therefore predicts almost no supply current except leakage.

Dynamic charging energy

An output and its connected wires/gates behave partly like capacitance. Charging an effective capacitance `C` from 0 to `VDD` draws energy from the supply. A commonly used switching-power estimate is:

```
Pdynamic ≈ α C VDD² f
```

where:

- `α` is the activity factor—how often the node makes an energy-consuming transition;
- `C` is effective switched capacitance;
- `VDD` is supply voltage;
- `f` is an associated switching/clock rate.

Conventions can move a factor between `α` and the energy-per-transition definition. The durable result is the quadratic dependence on supply voltage and linear dependence on switched capacitance and activity.

The energy stored on the charged node is:

```
Ecapacitor = ½ C VDD²
```

When the nMOS later discharges the node, that stored energy is mostly dissipated as heat in the pull-down path. The power supply also lost energy in the pull-up during charging.

Short-circuit current

During a finite input transition, nMOS and pMOS may both conduct. For a short interval there is a direct path:

```
VDD → pMOS → nMOS → GND
```

This **short-circuit current** adds energy beyond charging the load. It grows with conditions such as slow input edges and device sizing. It is not the same as leakage, and it does not imply a defective literal short circuit.

Leakage

When the gate is logically idle, real transistors still leak. Relevant mechanisms vary by process and operating point; a systems programmer mainly needs the consequence:

```
many transistors × small current each = meaningful idle power
```

Temperature and voltage affect leakage. Power-management features therefore include clock gating, power gating, voltage/frequency scaling, and low-power states. Clock gating reduces unnecessary switching; it does not remove all leakage. Power gating can reduce leakage in a domain but adds wake-up latency and state-retention concerns.

---

7. Delay: charge cannot move instantaneously

An inverter must charge or discharge its effective load:

```
load = transistor capacitances + wire capacitance + receiving gates
```

A useful first-order idea is:

```
delay grows when load capacitance grows
delay shrinks when available drive current grows
```

The HIGH-to-LOW and LOW-to-HIGH delays can differ because nMOS and pMOS drive strengths differ. A larger transistor can provide more current but also adds input and diffusion capacitance. Making every gate huge merely moves the problem upstream and consumes area and energy.

Fan-out is the load presented by downstream inputs. Gate inputs draw little steady DC current, but every input adds capacitance that must be charged and discharged. Thus “MOS inputs draw no current” cannot justify unlimited fan-out at speed.

This is enough device knowledge for the systems bridge:

```
longer wire / more gate inputs
    → more capacitance
    → more charge per transition
    → more delay and dynamic energy
    → lower feasible clock or extra pipeline stages/buffers
```

---

8. Optional lab — VTC, margins, current, and delay

The small lab is:

`04-cpu-and-chip-design/challenges/day-002-cmos-inverter-lab.cir`

It uses generic educational MOS models and performs:

1. a DC input sweep producing the inverter VTC, slope magnitude, and supply current;
2. a transient run producing input/output voltages and supply current;
3. propagation-delay and average-current measurements in the text log.

No GUI or external model file is required.

Predict

Record answers before running:

1. At `VIN = 0`, which device is on and where should `VOUT` settle?
2. At `VIN = VDD`, which device is on and where should `VOUT` settle?
3. Where should supply current be largest during the DC sweep?
4. During a rising input edge, does output rise or fall?
5. If output capacitance doubles, what happens to propagation delay and energy per full output transition?
6. Is static supply current in this model exactly representative of hardware?

Simulate

From the repository root:

```
mkdir -p /tmp/cmos-inverter-lab
ngspice -b \
  -o /tmp/cmos-inverter-lab/ngspice.log \
  04-cpu-and-chip-design/challenges/day-002-cmos-inverter-lab.cir
```

Outputs:

```
inverter-vtc.dat         VIN, VOUT, gain magnitude, supply current
inverter-transient.dat   time, VIN, VOUT, supply current
ngspice.log              measured levels, delay, and average current
```

These are whitespace-separated numeric text files. With `wr_singlescale`, ngspice writes one analysis scale followed by the requested vectors; the header names each column. They can be inspected without graphical tools:

```
less /tmp/cmos-inverter-lab/inverter-vtc.dat
less /tmp/cmos-inverter-lab/inverter-transient.dat
cat /tmp/cmos-inverter-lab/ngspice.log
```

Optional VTC plot:

```
gnuplot -persist <<'PLOT'
set xlabel "VIN (V)"
set ylabel "VOUT (V)"
set grid
plot "/tmp/cmos-inverter-lab/inverter-vtc.dat" \
  using 2:3 with lines title "CMOS inverter", \
  x with lines title "VOUT=VIN"
PLOT
```

Observe

Use the DC data to identify:

- output HIGH and LOW plateaus;
- the steep transition region;

- approximate points where gain magnitude crosses 1;
- the region of elevated supply current.

For this educational curve, estimate:

```
VIL = lower-VIN crossing where |dVOUT/dVIN| ≈ 1
VIH = upper-VIN crossing where |dVOUT/dVIN| ≈ 1
NML ≈ VIL - VOL
NMH ≈ VOH - VIH
```

State that these are estimates from this model, not guaranteed logic-family specifications.

Use the transient data and log to identify:

- finite input and output slopes;
- inversion;
- output delay after an input crossing;
- a current pulse during switching;
- asymmetry, if any, between rising and falling output delay.

Explain

Answer:

1. Why does the VTC have two plateaus and a steep middle?
2. Why can a degraded valid input produce a restored output near a rail?
3. Why is supply current larger in the transition region?
4. Why does capacitance affect delay although the next gate draws almost no steady input current?
5. Why can this lab teach mechanisms but not certify a CPU voltage limit?

---

9. From one inverter to CPU behavior

An inverter is not interesting because software frequently asks for logical NOT in isolation. It is interesting because complementary pull-up/pull-down networks generalize:

- NAND and NOR gates arrange series and parallel paths;
- larger gates implement terms used in decoders and multiplexers;
- gates compose into adders, comparators, register controls, and clock logic;
- clocked circuits capture restored levels as architectural and microarchitectural state.

Consider a machine instruction:

```
add x5, x6, x7
```

At the ISA level it means addition of register values. Below that abstraction:

```
instruction bits
  → decoder control voltages
  → mux paths select operands and ALU operation
  → carry and sum gate networks switch
  → many internal capacitances charge/discharge
  → result voltages settle
  → destination register captures the result on a clock edge
```

The software meaning “add” exists at the architectural layer. Individual transistors respond only to local voltages. Reliable restoration and timing allow those layers to coexist.

Why activity affects CPU power

A tight workload that repeatedly changes wide datapaths, caches, and execution units causes more nodes to switch than an idle core. More switching raises dynamic power. The relationship is not simply “one instruction costs one fixed energy”:

- instructions activate different units;
- operand values alter switching activity;
- cache misses engage different structures;
- out-of-order execution overlaps work;
- frequency and voltage may change;
- clock and power gating alter active domains.

Performance counters and model-specific energy interfaces can expose parts of this behavior later, but the physical starting point is still $C\ V^2$ energy associated with moving charge.

Why the CPU uses abstraction

Systems software cannot reason transistor-by-transistor. Hardware designers characterize gates and paths, impose timing constraints, and verify that valid outputs reach receiving registers under supported conditions. The ISA then promises architectural behavior to software.

That contract has boundaries. Overclocking, undervolting, overheating, or electrical faults can violate the assumptions beneath the ISA and produce incorrect execution. The abstraction is engineered, not magical.

---

10. Explain, Draw, Observe, Build

Explain

Without notes:

1. Explain both stable states of a CMOS inverter.
2. Explain why the transition region cannot be removed.
3. Distinguish transistor threshold, inverter transition, and logic input limits.
4. Derive NML and NMH from sending and receiving voltage guarantees.
5. Distinguish dynamic, short-circuit, and leakage power.
6. Explain restoration using charge, supply energy, and output rails.
7. Connect fan-out to capacitance, delay, and dynamic energy.

Draw

Draw one page containing:

```
CMOS inverter transistor network
        ↓
VTC with VOH, VOL, VIL, VIH
        ↓
noise margins
        ↓
two inverters showing restoration
        ↓
gate network → ALU path → destination register
```

Add arrows showing charge entering and leaving the output node.

Observe

If you run the optional lab, submit:

- predictions made before simulation;
- VTC and transient text files plus log;
- estimated `VIL`, `VIH`, `NML`, and `NMH`;
- observed propagation delays;
- one current observation;
- two reasons the numbers are model-specific.

If ngspice is unavailable, draw the expected VTC and transient waveforms, then annotate the same quantities. The conceptual checkpoint does not depend on simulator installation.

Build

Copy the lab into `/tmp`:

```
cp 04-cpu-and-chip-design/challenges/day-002-cmos-inverter-lab.cir \
  /tmp/cmos-inverter-lab/experiment.cir
```

Double `CLOAD`, predict both delay directions and switching current behavior, then rerun the copy. Explain the result with:

```
more capacitance → more charge required for the same voltage change
```

Do not tune transistor dimensions for an “optimal” circuit. The learning goal is the system consequence of load, not professional cell design.

---

11. Mastery checkpoint

Close the lesson and answer:

> How does a CMOS gate turn continuous, imperfect voltages into reliable logic that can carry an instruction through a CPU?

A complete answer should include:

- complementary pull-up and pull-down paths;
- stable outputs driven close to supply rails;
- a steep but finite VTC transition;
- defined input and output ranges;
- noise margins;
- gain and restoration using power-supply energy;
- capacitance and finite propagation delay;
- dynamic, short-circuit, and leakage power;
- timing constraints before a register captures state;
- the ISA as a higher-level contract built on these physical guarantees.

If the answer says “the transistor converts voltage to 0 or 1,” refine it. The circuit produces voltages; designers define and enforce the ranges that software can safely treat as bits.

Mental model at the end

```
Software requests an architectural operation.
The implementation causes selected electrical nodes to switch.
CMOS networks move charge toward a supply rail or ground.
Steep gate response restores valid but imperfect inputs.
Noise margins separate guaranteed output and input ranges.
Capacitance and finite current create delay and energy cost.
Registers capture settled logic under clock timing constraints.
The ISA hides this machinery while its guarantees hold.
```

Why? notebook

1. Why does complementary pull-up/pull-down operation reduce ideal static power?
2. Why are both devices partly on during many real transitions?
3. Why is a steep VTC useful for restoration?
4. Why are `VIL` and `VIH` different numbers?
5. Why is the interval between them not a third logic value?
6. Why does restoration need energy from the supply?
7. Why is restoration unable to recover a signal that crossed into the wrong valid range?
8. Why does doubling supply voltage increase dynamic energy by more than double?
9. Why can an “idle” chip still consume power?
10. Why does fan-out matter if MOS gates have negligible steady input current?
11. Why can lowering voltage save energy but reduce timing/noise headroom?
12. Where, along an instruction's path, do analog voltages stop mattering? Is there really one boundary?

References used selectively

Systems-oriented foundations:

- David A. Patterson and John L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, digital-logic and processor-datapath chapters. Used for the bridge from gates to datapath state and ISA-visible behavior.
- Neil H. E. Weste and David Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., chapters 2–4. Used selectively for inverter VTC, noise margins, capacitance, delay, and power—not as a fabrication or professional analog-design assignment.
- R. Jacob Baker, *CMOS: Circuit Design, Layout, and Simulation*, 4th ed., inverter chapters. Used to cross-check the complementary switching and VTC explanation.

Primary/current tool references:

- ngspice documentation and current manual for batch mode, DC/transient analyses, measurements, and text output: https://ngspice.sourceforge.io/docs.html
- ngspice source and release information: https://sourceforge.net/projects/ngspice/

Further credible context:

- IEEE IRDS reports, including system integration and power/performance scaling context: https://irds.ieee.org/editions
- Intel, *Intel 64 and IA-32 Architectures Software Developer's Manual*, Volume 1, for the architectural execution contract visible to systems software: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
- RISC-V International, unprivileged ISA specification, for another clear architectural contract independent of circuit implementation: https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html

The ngspice parameters are generic educational values, not a foundry model or a characterization of any commercial processor.

Next bridge: compose restoring CMOS gates into combinational networks, then study how capacitance, fan-out, and path depth become propagation delay that a clocked CPU must respect.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Cmos Inverter Lab

Format: ngspice circuit lab

What you will do: Day 2: CMOS inverter DC transfer and transient behavior

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
mkdir -p /tmp/day-002-cmos-inverter-lab && (cd /tmp/day-002-cmos-inverter-lab && ngspice -b
"$OLDPWD/04-cpu-and-chip-design/challenges/day-002-cmos-inverter-lab.cir")
```

- View lab source on GitHub

Day 3 — CMOS NAND, NOR, and Gate Networks

Target time: approximately 2–3 hours

- First-principles derivation and sketches: 75–90 minutes
- Small ngspice/Python lab: 35–45 minutes
- CPU bridge and mastery evidence: 30–40 minutes

> A truth table says what a gate must do. What arrangement of physical switches makes every row happen—and leaves a clean enough voltage for the next gate?

Why this day exists

Earlier we treated NAND and NOR as Boolean boxes. That was the correct level for combining gates, but it hid an important question:

> Where does the truth table exist in silicon?

It does not sit in a transistor as a tiny stored table. A MOS transistor reacts locally to voltages. The Boolean behavior emerges from a **network** of transistors that creates one conducting route for each required output state.

Today's path is:

```
input voltage ranges
    → MOS-controlled conducting paths
    → complementary pull-down and pull-up networks
    → restored NAND/NOR outputs
    → standard-cell gates
    → CPU decode, control, arithmetic, and timing paths
```

We will go only as deep as a systems programmer needs. The objective is not to design a fabrication process or size transistors. It is to understand why a gate has a particular topology, why high fan-in can be slower, and why a CPU's logical decisions ultimately involve charging and discharging real nodes.

---

1. The historical purpose: make switching logic composable

Relay networks made Boolean structure visible: series contacts express “all must conduct,” while parallel contacts express “either may conduct.” Relays were useful, but mechanical movement made them large, slow, noisy, and subject to wear. Vacuum tubes removed moving contacts but consumed substantial space and power.

The transistor supplied an electrically controlled solid-state device. Integrated circuits then placed many devices and their connections on one piece of material. The decisive gain was not merely replacing a relay with a smaller switch. Engineers could build **restoring, composable logic**:

```
one stage accepts imperfect but valid voltage ranges
    → computes through physical conduction
    → drives its output back toward a supply rail
    → the next stage receives another valid logic signal
```

This is why modern digital machines can contain enormous networks rather than one fragile chain of analog approximations.

Historical attribution needs care. Boole supplied an algebra for two-valued reasoning. Shannon showed how switching networks could be analyzed with that algebra. Semiconductor researchers developed practical transistors. CMOS logic and integrated-circuit manufacturing emerged through many later contributions. No one step, person, or device was “the CPU.”

---

2. The minimum MOSFET model we need

A MOSFET has more terminals and richer analog behavior than a switch. For this lesson, use a controlled-switch approximation:

nMOS

```
gate sufficiently HIGH relative to source → conducting path
gate LOW                                  → path mostly off
```

An nMOS device is therefore natural in a path that should conduct when its Boolean input is `1`.

pMOS

```
gate sufficiently LOW relative to source → conducting path
gate HIGH                                → path mostly off
```

A pMOS device is natural in a path that should conduct when its Boolean input is `0`.

The words “sufficiently,” “mostly,” and “relative” matter. Real MOSFET current depends continuously on terminal voltages. An off device leaks; an on device has finite resistance; terminals have capacitance; and switching is not instantaneous. Digital logic works by designing around those analog facts, not by making them disappear.

What not to say

Avoid:

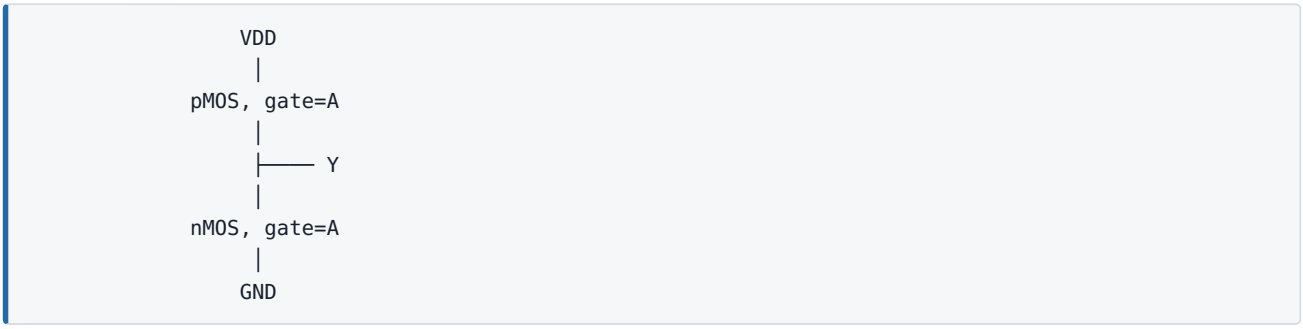
> One transistor is one Boolean gate.

A transistor is a controlled device. A conventional static-CMOS inverter uses two transistors. A two-input NAND uses four. More complex standard cells may contain larger networks. The Boolean function belongs to the **topology and electrical behavior of the whole circuit**, not to one isolated transistor.

---

3. Start with the CMOS inverter

A static-CMOS inverter has two complementary routes:



Treat `VDD` as the HIGH rail and ground as the LOW rail.

Case A = 0

- pMOS is on;
- nMOS is off;
- a route connects `Y` toward `VDD`;
- `Y` becomes HIGH.

Case A = 1

- pMOS is off;
- nMOS is on;
- a route connects `Y` toward ground;
- `Y` becomes LOW.

So the physical behavior matches:

A	Y
0	1
1	0

In a stable ideal logic state, there is no intended direct conducting path from `VDD` to ground. During a transition, both devices can briefly conduct; we will return to that as short-circuit power on Day 4.

Two networks, one output

Name the halves:

- **pull-down network (PDN):** nMOS devices that connect the output to ground exactly when the Boolean output must be `0`;

- **pull-up network (PUN):** pMOS devices that connect the output to `VDD` exactly when the Boolean output must be `1`.

For valid static inputs, complementary CMOS aims for exactly one of those networks to conduct. If neither conducts, the output can float. If both conduct strongly, the circuit has contention from supply to ground. Neither is the ordinary steady state we want.

---

4. Series and parallel are physical AND and OR of conduction

The easiest way to derive CMOS gates is to reason first about **conduction**, not the desired output expression.

Two nMOS devices in series:

```
Y ──[ nMOS A ]──[ nMOS B ]── GND
```

The complete path exists only when both devices conduct:

```
PDN conducts = A AND B
```

Two nMOS devices in parallel:

```
       ┌──[ nMOS A ]──┐
Y ─────┤              ├── GND
       └──[ nMOS B ]──┘
```

Either branch is sufficient:

```
PDN conducts = A OR B
```

For pMOS, LOW inputs turn devices on. The topology still obeys “series means all conducting paths are required; parallel means either conducting branch is enough,” but each pMOS's conduction condition is the inverted input.

This distinction prevents a common mistake. Series does not universally mean the gate's output is AND. Series describes when a **conducting path** exists. Whether that path pulls the output HIGH or LOW determines the final Boolean function.

---

5. Derive the two-input NAND

The target is:

```
Y = ¬(A ∧ B)
```

Do not begin by memorizing a picture. Ask when `Y` must be LOW.

```
Y = 0 exactly when A = 1 and B = 1
```

The nMOS pull-down must therefore conduct only for `A AND B`. Series nMOS devices provide that condition:

```
Y ──[ nMOS A ]──[ nMOS B ]── GND
```

Now derive the complementary pull-up. NAND is HIGH whenever at least one input is LOW. A pMOS controlled by A conducts when A is LOW, and a pMOS controlled by B conducts when B is LOW. Put those alternatives in parallel:

```
          ┌──[ pMOS A ]──┐
VDD ──────┤              ├── Y
          └──[ pMOS B ]──┘

Y ──[ nMOS A ]──[ nMOS B ]── GND
```

Trace all four stable cases:

A	B	Pull-up path	Pull-down path	Y
0	0	both pMOS branches conduct	series path broken	1
0	1	A pMOS branch conducts	series path broken	1
1	0	B pMOS branch conducts	series path broken	1
1	1	no pMOS branch conducts	complete series path	0

The topology and truth table agree.

Why NAND appears so often

NAND is functionally complete, but that is only part of the story. Its pull-down uses nMOS devices, and its pull-up offers parallel alternatives. Small-input NAND structures fit static CMOS naturally. Real cell libraries therefore offer several NAND variants along with many other gates.

This does **not** mean a synthesized CPU is literally a sea of identical two-input NAND symbols. Libraries include inverters, buffers, NAND/NOR, AND/OR, XOR, multiplexers, compound gates, latches, and flip-flops, each in several drive strengths. Synthesis and physical-design tools select cells to meet function, timing, power, and layout constraints.

---

6. Derive the two-input NOR

The target is:

```
Y = ¬(A ∨ B)
```

Again ask when the output must be LOW:

```
Y = 0 when A = 1 OR B = 1
```

Either HIGH input must create a pull-down route, so the nMOS devices are in parallel:

```
       ┌──[ nMOS A ]──┐
Y ─────┤              ├── GND
       └──[ nMOS B ]──┘
```

The output is HIGH only when both inputs are LOW. Both pMOS devices must conduct, so they are in series:

```
VDD ──[ pMOS A ]──[ pMOS B ]── Y

       ┌──[ nMOS A ]──┐
Y ─────┤              ├── GND
       └──[ nMOS B ]──┘
```

Trace it:

A	B	Pull-up path	Pull-down path	Y
0	0	complete series path	neither branch conducts	1
0	1	series path broken	B branch conducts	0
1	0	series path broken	A branch conducts	0
1	1	series path broken	both branches conduct	0

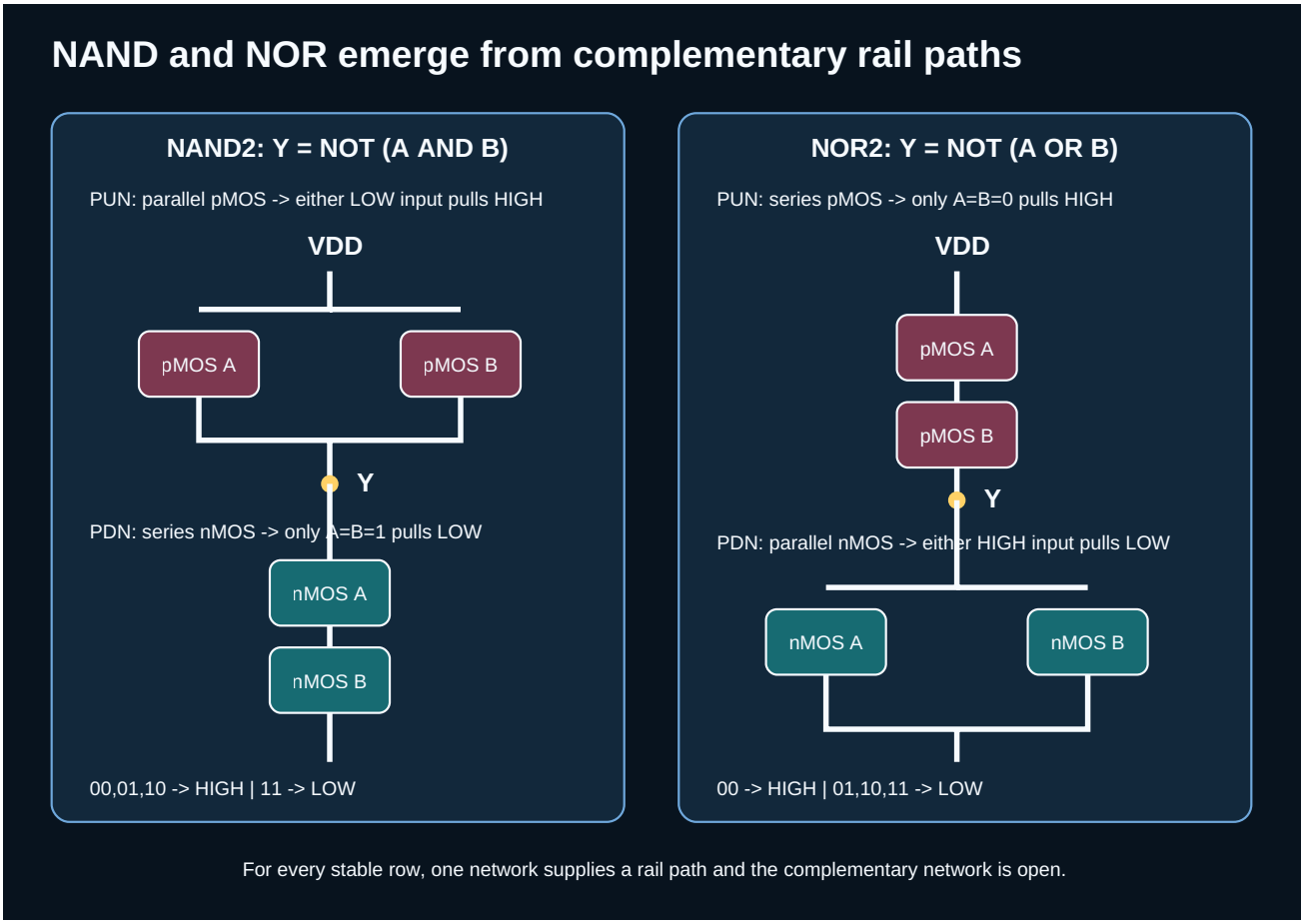
NAND and NOR are duals

Compare:

```
NAND: PDN series,   PUN parallel
NOR:  PDN parallel, PUN series
```

Replacing series with parallel while moving between pull-down and pull-up is the physical face of De Morgan's laws:

```
¬(A ∧ B) = ¬A ∨ ¬B
¬(A ∨ B) = ¬A ∧ ¬B
```



NAND and NOR exchange series and parallel topology between their complementary pull-up and pull-down networks.

---

7. Why static CMOS directly gives inverted functions

The nMOS network describes the condition under which the output is pulled LOW. If its conduction condition is `G`, then the gate output is:

```
Y = ¬G
```

That is why one-stage complementary networks naturally produce NAND, NOR, and inverting compound gates.

To obtain AND:

```
A, B → NAND → inverter → A ∧ B
```

To obtain OR:

```
A, B → NOR → inverter → A ∨ B
```

An AND symbol in a logic diagram may therefore map to more than one transistor stage, or the surrounding logic may absorb inversions so no separate AND cell is needed. Logical symbols, library cells, and transistor networks are related views—not interchangeable counts.

---

8. Derive a small compound network

Suppose the required output is:

```
Y = ¬(A ∧ (B ∨ C))
```

Step 1: derive the LOW condition

The output is LOW when:

```
A ∧ (B ∨ C)
```

Step 2: build the nMOS pull-down

- `A AND something` means a series requirement;
- `B OR C` means parallel alternatives.

```
Y ──[ nMOS A ]──┬──[ nMOS B ]── GND
                └──[ nMOS C ]── GND
```

Step 3: build the dual pMOS pull-up

Exchange series and parallel:

```
          ┌──[ pMOS A ]──────────────────┐
VDD ──────┤                              ├── Y
          └──[ pMOS B ]──[ pMOS C ]──────┘
```

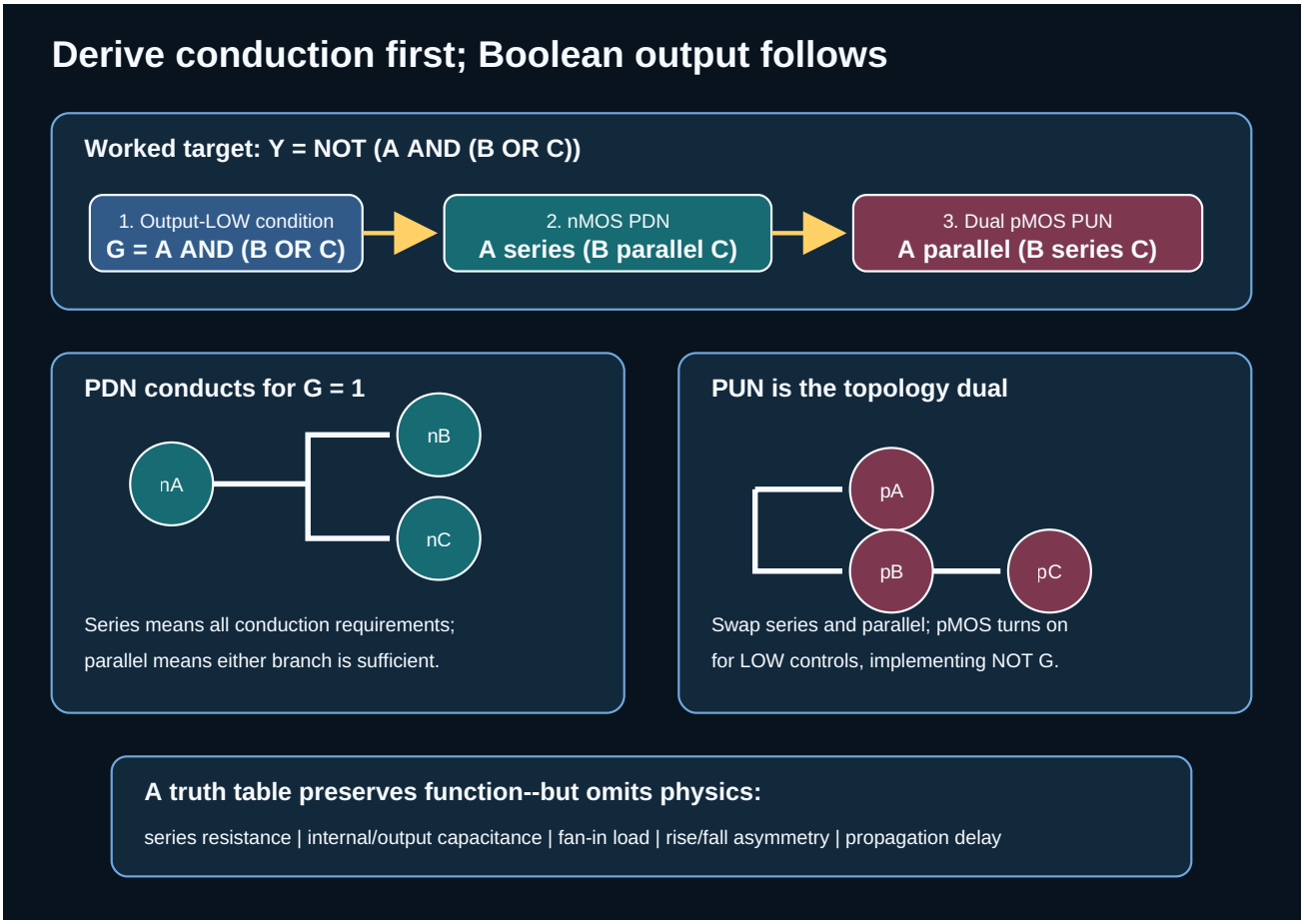
The top branch conducts if A is LOW. The lower branch conducts if both B and C are LOW. Therefore the output is HIGH for:

```
¬A ∨ (¬B ∧ ¬C)
```

By De Morgan:

```
¬(A ∧ (B ∨ C)) = ¬A ∨ (¬B ∧ ¬C)
```

The two derivations agree.



Deriving the output-LOW conduction condition exposes topology, while the truth table omits resistance, capacitance, fan-in, and delay.

A reusable derivation procedure

For an inverting static-CMOS function:

1. write the condition that should make the output `0`;
2. use nMOS series for AND and nMOS parallel for OR to build the PDN;
3. construct the PUN as the dual network—exchange series and parallel and use pMOS devices controlled by the same inputs;
4. enumerate every input row and check that exactly one network provides the intended rail path in the stable state.

This is enough depth to read a simple schematic and understand the physical origin of a logic function.

---

9. Series resistance, parallel paths, and fan-in

An on transistor is not an ideal wire. It has finite effective resistance. If two conducting devices lie in series, the current encounters both:

```
rough intuition: Rpath ≈ R1 + R2
```

The approximation is deliberately simple—the actual device current is nonlinear—but it gives the right systems-level consequence. A longer series stack generally drives a capacitive output less strongly and takes longer to move its voltage.

Parallel paths can provide alternative current routes. Yet the unused devices, internal diffusion regions, wiring, and receiver gates all add capacitance. More devices are not free.

Fan-in is the number of inputs accepted by a gate. Increasing fan-in tends to bring:

- longer series stacks in one complementary network;
- more capacitance at the output and internal nodes;
- more capacitance presented to input drivers;
- slower worst-case transitions;
- a larger and harder-to-route cell.

An eight-input Boolean condition is therefore not automatically best implemented as one enormous eight-input gate. A library and synthesis tool may use a tree of smaller gates. The tree adds stages but avoids one weak, high-capacitance structure. Timing is an electrical-path question, not simply “fewest Boolean symbols wins.”

Do not turn this into transistor-sizing arithmetic here. The durable model is:

```
more series resistance + more node capacitance → more transition time
```

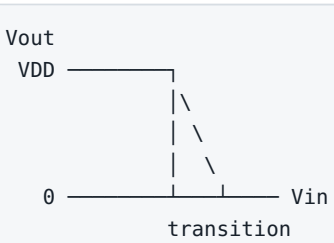
Day 4 will make that relationship observable.

---

10. Restoration: why many stages can be connected

Suppose a HIGH input is not exactly at `VDD`, or noise shifts it slightly. If every stage merely copied an analog fraction, errors would accumulate down a long path.

A CMOS inverter has a steep transfer region:



Outside the transition region, the output is driven near a rail. Near the switching point, a small input change produces a larger opposite output change. This voltage gain is what makes the stage **restoring**.

Digital systems then define ranges:

```
input LOW range     safely interpreted as 0
uncertain region    behavior not guaranteed as a logic value
input HIGH range    safely interpreted as 1
```

The separation between a guaranteed output level and the receiving input threshold gives a **noise margin**. We do not need to calculate exact margins today. We need the consequence: a valid but imperfect output should still be interpreted correctly and restored by the next stage.

Restoration does not make circuits immune to everything. Excessive supply noise, crosstalk, slow edges, process variation, radiation-induced charge, or violated timing assumptions can still cause failure. Reliability comes from quantified margins and verification, not from the word “digital.”

---

11. Where these networks appear in a CPU

Consider one decode condition:

```
is_integer_add =
    opcode_matches
    AND privilege_allows
    AND operands_ready
    AND no_flush
```

At the ISA level, this is a condition about instruction meaning. At the RTL level, it is a Boolean expression. After synthesis, it becomes library cells and wires. Inside those cells are transistor networks. While the CPU runs, capacitances on those nodes charge and discharge.

```
instruction bits and machine state
    → decode/control Boolean network
    → voltage transitions through cells and wires
    → enable/select signals
    → register, ALU, or memory action
```

The transistors do not understand `ADD`, privilege, or readiness. Their local electrical behavior happens to implement a network whose abstraction matches those predicates.

The same chain appears in:

- ALU carry and selection logic;
- branch-condition evaluation;
- address-generation control;
- cache tag comparison and hit selection;
- pipeline valid, stall, flush, and forwarding logic;
- interrupt and exception prioritization.

Once topology introduces finite resistance and each node has capacitance, these decisions take time. That is the bridge from “the decoder computes a Boolean answer” to “the clock cannot be arbitrarily fast.”

---

12. Lab — observe topology and restoration

The lab contains:

- a two-input static-CMOS NAND;
- a two-input static-CMOS NOR;

- an inverter swept through all input voltages;
- documented, generic MOS level-1 models;
- a Python runner that checks truth rows and summarizes restoration.

Files:

- `04-cpu-and-chip-design/challenges/day-003-cmos-networks.cir`
- `04-cpu-and-chip-design/challenges/day-003-cmos-networks.py`

Predict before running

Write down:

1. For `(A,B) = (0,0), (1,0), (0,1), (1,1)`, which NAND network conducts?
2. For the same rows, which NOR network conducts?
3. At `Vin = 0`, should the inverter output settle near `0`, halfway, or `VDD`?
4. Near the inverter switching boundary, should the transfer curve be flat or steep?

Run

From the repository root:

```
python3 04-cpu-and-chip-design/challenges/day-003-cmos-networks.py
```

To retain the raw ngspice data and log:

```
python3 04-cpu-and-chip-design/challenges/day-003-cmos-networks.py \
  --keep /tmp/day-003-cmos-evidence
```

The `.cir` file is also directly batch-runnable:

```
repo=$(pwd)
tmpdir=$(mktemp -d)
cd "$tmpdir"
ngspice -b \
  "$repo/04-cpu-and-chip-design/challenges/day-003-cmos-networks.cir"
```

Use the Python command for the normal course workflow because it creates a temporary directory, checks expected values, and leaves the repository clean.

What the model does—and does not prove

The netlist uses the classical ngspice MOS level-1 square-law model with explicit teaching parameters. It is intentionally:

- public and inspectable;
- small enough to connect topology to waveforms;
- adequate for qualitative NAND/NOR and restoration observations.

It is **not**:

- a model for a named fabrication process;
- accurate for a modern nanometre CPU transistor;
- a standard-cell characterization;

- evidence of real CPU voltage, delay, power, or noise margin;
- suitable for tape-out or reliability sign-off.

Modern process models include short-channel, leakage, variation, interconnect, and many other effects absent here. The lab proves conceptual relationships inside its stated model.

Explain the result

After running, complete these sentences:

1. NAND is LOW only for `11` because ...
2. NOR is HIGH only for `00` because ...
3. The inverter restores logic levels because ...
4. A high-fan-in gate can slow a CPU path because ...

---

13. Evidence: Explain, Draw, Observe, Connect

Explain

Without notes:

1. Why is “one transistor equals one gate” wrong?
2. What are the roles of the PUN and PDN?
3. Why does a NAND use series nMOS and parallel pMOS?
4. Why does a NOR use parallel nMOS and series pMOS?
5. Why do AND and OR often require another inversion in static CMOS?
6. Why can high fan-in make a gate slower?
7. What does restoration contribute that a passive switch network does not?

Draw

Draw from memory:

1. CMOS inverter;
2. NAND2 PUN and PDN;
3. NOR2 PUN and PDN;
4. the compound gate `¬(A ∧ (B ∨ C))`.

For every drawing, annotate which input combinations make each network conduct. A transistor sketch without the conduction explanation is incomplete.

Observe

Keep:

- the four settled NAND/NOR rows;
- inverter output at both rails;
- the approximate switching region;
- one sentence explaining why a steep transfer matters.

Connect

Choose one CPU condition—branch taken, cache hit, pipeline stall, or register write enable—and trace:

```
architectural meaning
    → Boolean predicate
    → gate network
    → pull-up/pull-down conduction
    → restored output voltage
    → next stage
```

Do not claim a particular commercial CPU uses your exact network. The point is the abstraction bridge.

---

14. Mastery checkpoint

Close the lesson and answer:

> Starting from a Boolean function, how can a complementary CMOS network produce a restored output, and why does its topology matter to CPU timing?

A complete answer should include:

- nMOS and pMOS conduction conventions;
- PDN for the output-LOW condition;
- dual PUN for the output-HIGH condition;
- series as “all paths required” and parallel as “either path sufficient”;
- NAND and NOR derivations;
- finite series resistance and node capacitance;
- restoration toward supply rails;
- the connection from cell delay to a CPU logic path.

If the answer is only “transistors act like switches,” it is not complete.

Why? notebook

1. Why derive the pull-down from the output-LOW condition?
2. Why must the pull-up be complementary rather than an unrelated network?
3. Why does series describe conduction AND but not automatically output AND?
4. Why do NAND and NOR emerge directly as inverting static-CMOS gates?
5. Why can fewer logic symbols still produce a slower physical circuit?
6. Why does a floating output violate the model expected by the next gate?
7. Why is a steep inverter transfer useful but not a guarantee against every disturbance?
8. Where does an instruction's meaning disappear as we descend toward transistors?

Mental model at the end

```
A transistor responds to terminal voltages.
A network creates or removes conducting paths.
The PDN realizes the condition for output LOW.
The dual PUN realizes the condition for output HIGH.
The output is actively restored toward a rail.
Series stacks add effective resistance; nodes add capacitance.
Gate networks implement CPU predicates without understanding them.
```

References used selectively

- Claude E. Shannon, “A Symbolic Analysis of Relay and Switching Circuits,” 1937 thesis, for the historical algebra-to-switching-network bridge: https://dspace.mit.edu/handle/1721.1/11173
- Computer History Museum, “The Silicon Engine,” for the historical transition from relay/vacuum-tube limitations to transistor and integrated-circuit technologies: https://www.computerhistory.org/siliconengine/
- Hans-Dieter Wacker, *Introduction to Digital Circuits*, “Basic Digital Circuits,” for complementary static-CMOS PUN/PDN derivations and fan-in consequences: https://www.strumpen.net/dc/build/html/basiccircuits/basiccircuits.html
- Holger Vogt et al., *ngspice User's Manual*, Version 47, 11 August 2026, MOSFET device and level-1 model sections: https://ngspice.sourceforge.io/docs/ngspice-html-manual/manual.xhtml
- ngspice manual, “Notes on Level 1–6 models,” for the documented limits and parameters of the classical models used by the lab: https://nmg.gitlab.io/ngspice-manual/mosfets/mosfetmodels_nmos_pmos/notesonlevel1-6models.html

Next bridge: a logically correct output still takes time to move. Day 4 connects capacitance and finite drive to propagation delay, fan-out, power, PVT variation, and the CPU's critical path.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Cmos Networks

Format: Python lab

What you will do: Run and summarize the Day 3 static-CMOS ngspice experiment.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
python3 04-cpu-and-chip-design/challenges/day-003-cmos-networks.py
```

- View lab source on GitHub

Day 4 — Delay, Capacitance, Fan-out, and Power

Target time: approximately 2–3 hours

- First-principles timing and power model: 75–90 minutes
- ngspice/Python load sweep: 35–45 minutes
- CPU timing, reliability, and evidence: 35–45 minutes

> The Boolean answer may be correct. What determines whether it arrives before the next clock edge?

Why this day exists

A truth table has no time axis. It says that an inverter maps `0` to `1`, but not whether the output changes in one picosecond or one second.

A CPU does not receive unlimited time. A pipeline stage launches signals, combinational logic responds, and a later register must capture the settled result. If the slowest allowed path has not completed, the machine can store the wrong bit even though every gate has the correct truth table.

Today's path is:

```
every node stores charge
    + every driver has finite current/resistance
    → transitions take time
    → load and fan-out alter delay
    → switching consumes energy
    → process, voltage, and temperature alter both
    → the worst valid path constrains the clock
```

We will keep the hardware depth purposeful. You should finish able to reason about CPU timing, performance, power, and reliability—not perform analog transistor optimization or standard-cell characterization.

---

1. The historical problem: integration made time and power first-class

Early switching machines were visibly slow: relay contacts moved; vacuum tubes and long wires had finite response. Transistors and integrated circuits made switching dramatically faster and packed far more logic together, but they did not make delay vanish.

Integration changed the scale of the engineering problem:

- each individual node might carry tiny capacitance;
- billions of devices and extensive wiring create enormous aggregate load;
- higher clock rates cause more charge movement per second;

- heat removal and supply delivery limit sustained operation;
- variation means the nominally “same” path is not identical on every chip or under every operating condition.

This is why modern processor design cannot be explained by Boolean algebra alone. A design must be logically correct **and** close timing under allowed conditions **and** stay within power, temperature, and reliability limits.

The historical “frequency scaling” story also needs precision. Higher clock rates once delivered large performance gains, but power density, wire delay, memory behavior, and diminishing voltage scaling made indefinite frequency growth impractical. Modern CPUs use pipelining, parallel execution, caches, specialized units, power management, and many cores because “just raise the clock” is not a free strategy.

---

2. A logic node is a physical capacitor

Where does capacitance come from in a gate network?

- MOS gate terminals store charge;
- transistor junctions and internal nodes contribute capacitance;
- nearby and long wires contribute capacitance;
- every receiving gate attached to an output adds input capacitance;
- package and off-chip connections can add far larger loads.

We often combine those effects into an effective load:

```
Cload = receiver input capacitance
      + wire capacitance
      + driver/output parasitics
      + other coupled contributions represented by the model
```

The precise decomposition belongs to circuit and physical design. For systems reasoning, the important relationship is:

```
Q = C × V
```

To move a capacitance `C` through a voltage change `V`, the driver must move charge `Q`. More capacitance or a larger voltage swing requires more charge.

The output does not teleport between rails

When a CMOS gate output rises, its pull-up network supplies current and charges the node. When it falls, the pull-down network removes charge toward ground.

```
rise: VDD → finite pull-up path → Cload
fall: Cload → finite pull-down path → GND
```

Because current is finite, both changes take time.

---

3. RC delay: a useful model, not a transistor law

Approximate the conducting network by an effective resistance `R` and the output by a capacitance `C`:

```
VDD ── R ── output
             |
             C
             |
            GND
```

For a simple charging RC network:

```
Vout(t) = VDD × (1 - e^(-t/RC))
```

For discharge:

```
Vout(t) = Vinitial × e^(-t/RC)
```

The product:

```
τ = R × C
```

is the time constant. After one `τ`, a charging node has traversed about 63% of the final change. The familiar factor `0.69RC` estimates the time to cross 50% in a first-order model.

Do not promote that approximation into a universal gate-delay formula. A real MOSFET is nonlinear; the effective resistance changes during a transition; internal nodes, input slew, coupling, and wires matter. Characterized cell libraries and timing tools use richer data. `RC` remains valuable because it explains the direction of effects:

```
larger effective R → slower transition
larger effective C → slower transition
```

Day 3's series transistor stack raises effective path resistance. Today's receiver and wire load raises capacitance. Together they explain why topology and connectivity affect timing.

---

4. Propagation delay is measured between thresholds

An input edge itself takes time, and the output does not begin and finish at one instant. We therefore define reproducible landmarks.

High-to-low propagation delay

`tpHL` is commonly measured from the input crossing 50% of its swing to the output crossing 50% while falling:

```
input:    ___/‾‾‾‾‾
               | 50%
output:   ‾‾‾‾\____
                  | 50%
            <----->
               tpHL
```

Low-to-high propagation delay

`tpLH` is the corresponding input-to-output delay for a rising output.

One may summarize a gate with an average propagation delay:

```
tpd ≈ (tpHL + tpLH) / 2
```

or use the worse direction when a bound is needed. Always check what a quoted number means. “Gate delay” without measurement conditions, input slew, load, supply, temperature, and process assumptions is incomplete.

Rise and fall time

Rise time and fall time describe the output transition itself, often between 10% and 90%:

```
trise: output crosses 10% → output crosses 90%
tfall: output crosses 90% → output crosses 10%
```

Propagation delay and transition time are related but not identical:

- propagation delay asks when the output response reaches a reference point;
- rise/fall time asks how long the output spends traversing most of its swing.

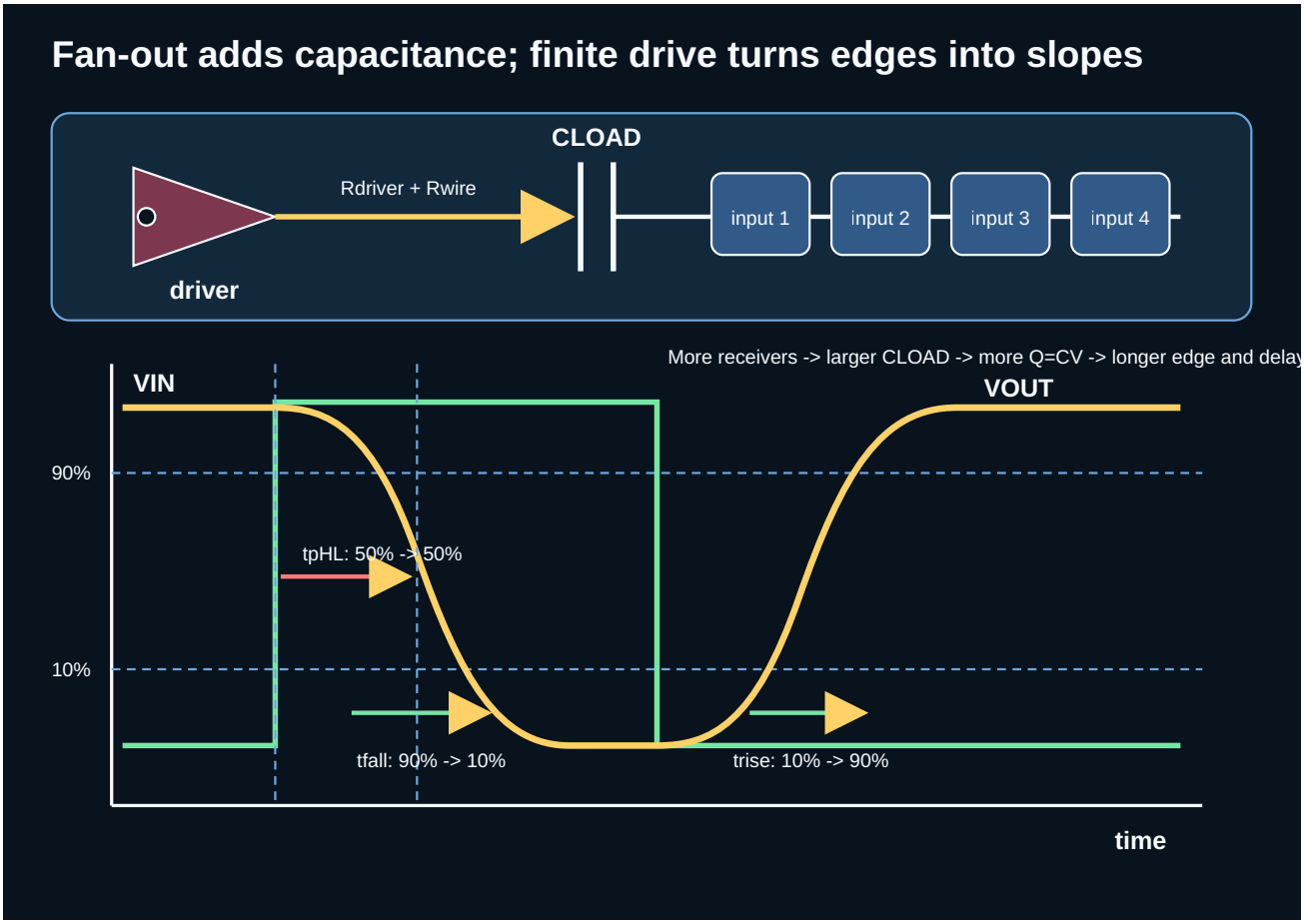
A slow edge can make the next stage slower and can leave its transistors simultaneously conducting for longer. Timing analysis therefore tracks both arrival time and slew.

Why rise and fall differ

Pull-up and pull-down networks use different device types and topologies. Their drive need not match exactly. NAND and NOR also have different worst-case stacks in the rising and falling directions. Consequently:

```
tpHL need not equal tpLH
trise need not equal tfall
```

The difference is physical, not a violation of the truth table.



Fan-out increases capacitive load, stretching both threshold-to-threshold propagation delay and the output edge itself.

---

5. Fan-out turns one decision into a load problem

Fan-out informally means how many receiving inputs an output drives:

```
                ┌→ gate input
driver output ──┼→ gate input
                ├→ gate input
                └→ gate input
```

Each receiver contributes gate capacitance. The connecting wire adds more. So increasing fan-out generally increases `Cload`, which increases charge, transition time, delay, and switching energy.

The integer receiver count is only a rough description. Four tiny nearby inputs and four large distant inputs are not the same electrical load. Timing tools work with capacitance, slew, wire estimates or extracted parasitics, and characterized cell behavior—not receiver count alone.

Why not attach everything to one signal?

A control signal such as clock enable, reset, or pipeline valid may need to reach many destinations. One weak source driving the entire load would switch slowly. Designers use buffering trees, replication, and structured distribution:

```
source → buffer ┬→ buffer → local receivers
                └→ buffer → local receivers
```

Extra stages sound slower, but each stage drives a manageable load. Under a large total load, a staged path can arrive sooner than one small gate trying to move all the charge.

Logical effort—intuition only

Logical effort is a compact way to reason about two facts:

1. some gate topologies are intrinsically harder to drive than an inverter with comparable output capability;
2. delay also depends on the external capacitive load relative to the gate's own input capacitance.

It helps designers compare topology and staging without beginning every question from a full transistor simulation. For this course, retain only:

```
gate topology contributes intrinsic difficulty
load contributes electrical effort
too much load on one stage is slow
staging can distribute a large load
```

We will not calculate logical-effort values or perform transistor-sizing exercises. CPU timing intuition—not analog design methodology—is the goal.

---

6. Dynamic energy: derive $C\ V^2$

To charge a capacitor from 0 to V, the supply moves charge:

$$Q = C\ V$$

The supply operates at voltage V, so the energy drawn from it during a low-to-high charge is:

$$E_{supply} = V \times Q = C\ V^2$$

An ideal capacitor stores:

$$E_{cap} = \tfrac{1}{2}\ C\ V^2$$

Where did the other half go? In the simple resistive charging model, it is dissipated in the charging path. When the node later discharges to ground, the stored half is also dissipated. Thus a complete $0 \to 1 \to 0$ activity cycle draws approximately:

$$E_{cycle} = C\ V^2$$

If this happens at an average switching rate αf:

$$P_{dynamic} \approx \alpha\ C\ V^2\ f$$

where:

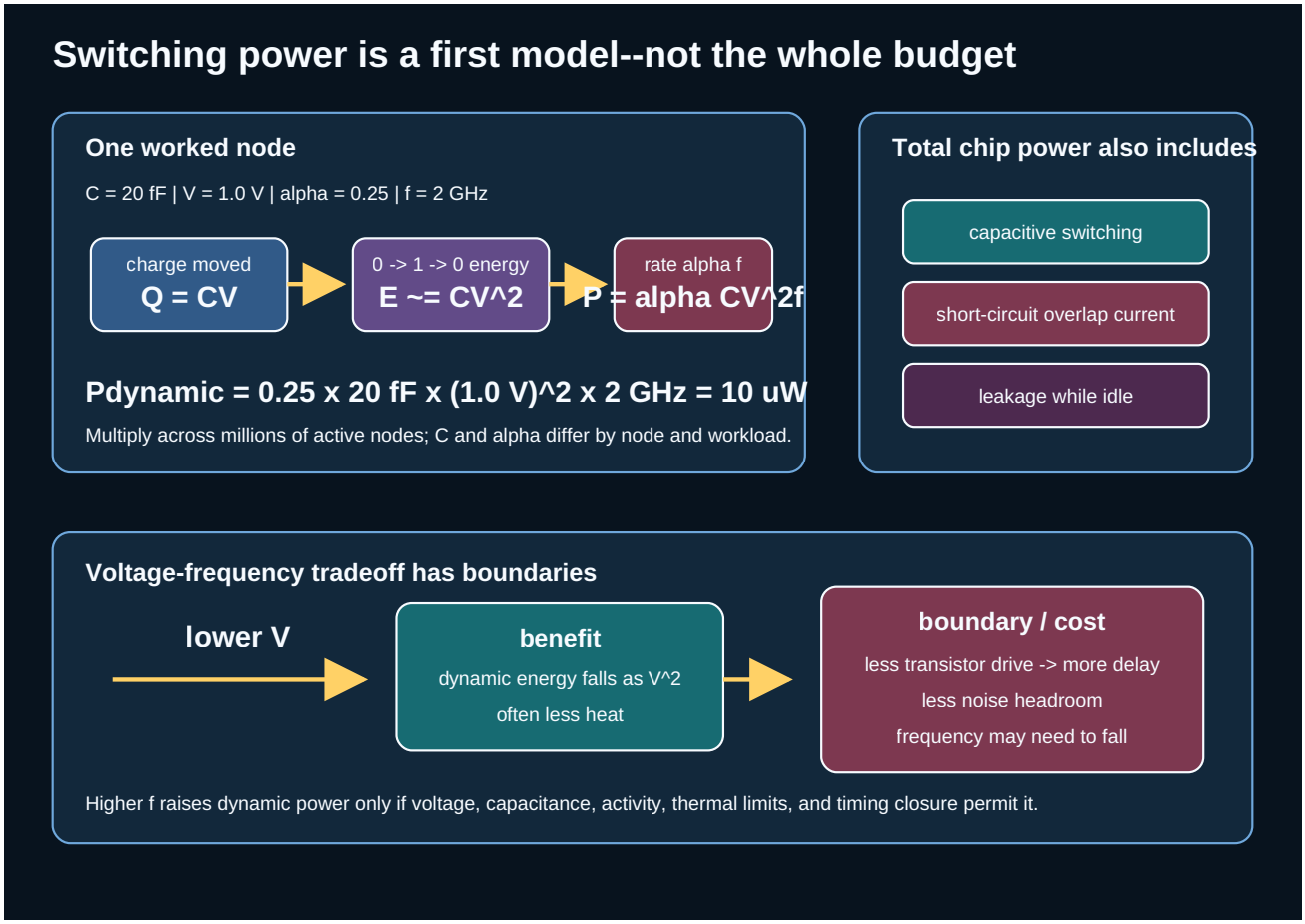
- `C` is effective switched capacitance;
- `V` is supply voltage;
- `f` is a reference frequency, often clock frequency;
- `α` is an activity factor connecting that reference to actual switching.

Activity-factor conventions vary. State whether `α` counts low-to-high charges, transitions, or cycles before comparing formulas or numbers.

What the equation tells a systems programmer

- more active hardware raises power;
- larger loads and wires raise power;
- higher frequency raises switching power roughly linearly if activity is otherwise unchanged;
- voltage is especially expensive because of the square;
- reducing work, toggles, capacitance, or voltage can matter more than a source-level instruction count suggests.

Voltage reduction also slows transistors and reduces noise margin, so dynamic-voltage/frequency scaling is a performance-reliability trade, not a free energy switch.



The worked switching estimate is only one power component, and lower voltage trades energy against delay and noise headroom.

---

7. Dynamic power is not all power

Short-circuit power

During a finite input transition, the pMOS and nMOS networks can both conduct briefly:

```
VDD → partially conducting pull-up
    → output network
    → partially conducting pull-down → GND
```

That current does not exist in the ideal stable-state truth table. Slower input edges can lengthen the overlap interval. The exact contribution depends on the cell, load, slew, and supply.

Leakage power

“Off” transistors are not perfect open switches. Leakage includes several physical mechanisms, and their importance depends strongly on process and operating conditions. A useful system-level model is:

```
Pleak ≈ VDD × Ileak
```

Leakage is paid even when useful nodes are not switching. That matters for:

- large transistor counts;
- idle cores and caches;
- thermal management;
- battery life;
- the choice to clock-gate, power-gate, or place blocks in lower-power states.

Clock gating reduces unnecessary switching but does not remove all leakage. Power gating can reduce leakage more aggressively but adds state-retention, wake-up, supply-integrity, and latency concerns.

Internal switching and glitches

A combinational network can briefly pass through intermediate states when different paths arrive at different times. Those **glitches** may charge and discharge internal nodes even if the final Boolean output does not change. They consume dynamic energy and, if they escape an allowed timing window, may affect behavior.

So `α C V² f` is a powerful first model, not a complete power report.

---

8. PVT: the same design does not have one delay

Timing depends on **process, voltage, and temperature (PVT)**.

Process

Manufacturing variation changes device properties and interconnect. Some instances or regions may switch faster or slower than nominal assumptions. Design flows therefore analyze characterized corners and statistical or on-chip variation models appropriate to the technology.

Voltage

Lower supply voltage generally reduces transistor drive and increases delay. Local voltage droop can make a path temporarily slower exactly when substantial activity draws current. Higher voltage can improve speed within limits but raises dynamic power and electrical stress.

Temperature

Temperature changes mobility, threshold behavior, leakage, and wire resistance. The direction and size of net delay change depend on technology, voltage, and circuit. Do not memorize “hot is always slower” as a universal device law, though slow/hot assumptions are common in simplified examples.

Why corners are a reliability requirement

A CPU sold to operate across a specified range must work outside a room- temperature typical simulation. Timing sign-off asks whether all constrained paths meet requirements across defined operating scenarios, not merely whether one nominal waveform looks clean.

Day 4's lab perturbs simple model parameters to expose sensitivity. Those perturbations are **illustrative corners**, not a replacement for foundry models or sign-off analysis.

---

9. From one gate delay to a CPU critical path

Between two state elements, a synchronous path is conceptually:

```
launch register
    → clock-to-Q delay
    → combinational gates and wires
    → setup requirement at capture register
```

A simplified setup constraint is:

```
Tclock ≥ tclock-to-Q + tcombinational,max + tsetup
         + clock/skew/jitter/variation margin
```

The **critical path** is the path with the least timing slack under the relevant analysis—not necessarily the one with the most gate symbols. Connectivity, cell choices, transition slew, wire length, fan-out, coupling, variation, and clock relationships all matter.

Concrete CPU candidates

Depending on microarchitecture, long paths may include:

- instruction decode and control selection;
- ALU carry or compare logic;

- bypass/forwarding selection;
- branch condition and next-PC selection;
- load address generation and cache tag/data selection;
- wide priority encoders or wake-up/select logic.

There is no universal “the CPU critical path.” It can change with design revision, operating mode, PVT corner, placement, or optimization.

What if the path is too slow?

Possible responses include:

- restructure Boolean logic;
- reduce fan-out or replicate a driver;
- insert buffers;
- place communicating cells closer;
- choose a stronger or faster characterized cell;
- reduce wire load;
- add a pipeline stage;
- lower the clock frequency;
- raise voltage within allowed limits;
- change the microarchitecture.

Every response has costs. A pipeline stage can shorten combinational work per cycle and raise potential throughput, but adds registers, clock power, latency, control complexity, and branch-recovery cost. Stronger cells can reduce delay but increase input capacitance, area, and power.

Frequency is not performance

The clock bound gives:

```
fmax ≈ 1 / Tmin
```

but program performance also depends on:

- instructions or useful operations completed per cycle;
- stalls and dependencies;
- branch prediction;
- cache and memory behavior;
- core count and parallelism;
- thermal and power limits;
- operating frequency actually sustained.

A deeper pipeline with a higher advertised clock can lose work on mispredictions or consume more power. Timing is one constraint in performance, not a complete performance metric.

---

10. Timing failures become architectural failures

If a signal arrives after the capture window, a register may capture the old value, the new value, or—in edge cases we will study later—enter metastable behavior. Above that physical event, the symptom might look like:

- a wrong arithmetic result;
- a missed pipeline stall;
- an incorrect branch target;
- a corrupted address;
- an invalid cache-control action;
- an exception that appears unrelated to the underlying path.

Software expects the hardware contract to hold. It cannot retry every internal gate transition. Hardware therefore uses design margins, static timing analysis, dynamic simulation, physical verification, on-chip monitors, clock/voltage control, testing, and error-detection mechanisms.

A feedback loop worth remembering

```
more switching
    → more power
    → higher temperature / more supply droop
    → changed delay and leakage
    → reduced timing margin or throttling
    → changed delivered performance
```

This is why `cpufreq`, thermal throttling, power limits, boost behavior, and hardware error reports are connected to transistor-level constraints even though Linux exposes them through high-level interfaces.

---

11. Lab — sweep load, observe delay and energy

The lab uses one CMOS inverter and varies an explicit output capacitor. For discussion, `4 fF` is labeled as one notional receiver input:

```
fan-out 1 → 4 fF
fan-out 2 → 8 fF
fan-out 4 → 16 fF
fan-out 8 → 32 fF
```

That mapping is a teaching convention, not a real cell-library specification.

Files:

- `04-cpu-and-chip-design/challenges/day-004-delay-power.cir`
- `04-cpu-and-chip-design/challenges/day-004-delay-power.py`

The script measures:

- `tpHL` and `tpLH` at 50% crossings;
- 10–90% rise and fall time;
- supply energy over one complete input period;

- three small PVT-sensitivity cases at one load.

Predict before running

Write down:

1. As load doubles, what should happen to propagation delay?
2. What should happen to rise and fall time?
3. What should happen to supply energy per toggle?
4. Will rise and fall measurements be identical?
5. Which should be slower in the lab's illustrative set: `slow/low/hot` or `fast/high/cool`?
6. Which of these predictions are qualitative, and which claim a numerical relationship?

Run

From the repository root:

```
python3 04-cpu-and-chip-design/challenges/day-004-delay-power.py
```

The script is batch-runnable, needs no plotting package, and writes:

```
/tmp/day-004-delay-power.csv
```

Run one nominal circuit directly if you want the raw ngspice log and waveform:

```
repo=$(pwd)
tmpdir=$(mktemp -d)
cd "$tmpdir"
ngspice -b \
  "$repo/04-cpu-and-chip-design/challenges/day-004-delay-power.cir"
```

The direct run writes `day-004-waveform.dat` in the temporary directory.

Read the result conservatively

Expected qualitative observations:

- added load increases propagation delay;
- added load lengthens the edge;
- supply energy rises with switched capacitance;
- pull-up and pull-down results differ somewhat;
- changed model strength, voltage, and temperature alter delay.

Do not claim:

- that the delay belongs to any commercial CPU;
- that one notional fan-out has a universal capacitance;
- that the sweep establishes a modern process corner;
- that the level-1 model predicts nanometre-device leakage or short-channel behavior;
- that a four-point curve proves exact linearity under all loads.

Connect every row to a CPU

For each increasing-load row, say:

> If this node were on a pipeline's limiting path, the added delay would consume timing slack. The design might need less load, buffering, restructuring, another pipeline stage, or a longer clock period.

For the energy column, say:

> If many such nodes toggle repeatedly, their individual charging events accumulate into dynamic CPU power and heat.

For the PVT rows, say:

> A clock safe at nominal conditions is not sufficient evidence for the full operating range.

---

12. Why this model is intentionally limited

The `.cir` file uses documented ngspice level-1 MOS models with explicit generic parameters. It includes a simple charge-storage model and explicit load capacitance so delay and energy trends are visible.

It omits or greatly simplifies:

- modern short-channel transistor behavior;
- real process variation and correlated on-chip variation;
- accurate leakage mechanisms;
- extracted metal resistance, capacitance, and coupling;
- cell-internal characterization across input slew and output load;
- clock-tree and power-grid models;
- package effects and supply noise;
- aging, electromigration, and long-term reliability;
- statistical timing and sign-off constraints.

The numeric results belong only to this netlist and simulator configuration. The conceptual result—finite drive charging capacitance causes delay and energy—is the evidence we need.

ngspice 47 is the validation target for these files. The netlists use public syntax and no proprietary model cards.

---

13. Evidence: Explain, Draw, Observe, Connect

Explain

Without notes:

1. Where does capacitance exist in a logic path?
2. Why does `Q = CV` imply that a heavier load switches more slowly?
3. Distinguish propagation delay from rise/fall time.
4. Why is fan-out count only an approximation of load?
5. Derive the qualitative meaning of $P \approx \alpha CV^2 f$.
6. Distinguish capacitive switching, short-circuit, and leakage power.
7. Why must timing be checked across PVT conditions?
8. Why is the critical path not necessarily the path with the most gates?

Draw

Draw:

```
launch register
    → gate/wire RC path
    → high-fan-out node
    → capture register
```

Annotate:

- clock-to-Q;
- propagation delays;
- output load;
- setup time;
- clock period;
- one place where buffering could help;
- one place where extra buffering would add power.

Observe

Keep the CSV and record:

- delay at fan-out 1 and fan-out 8;
- rise/fall time change over the sweep;
- energy change over the sweep;
- fastest and slowest illustrative corner;
- one asymmetry between rising and falling behavior.

Connect

Choose one:

- ALU result to destination register;
- branch compare to next-PC register;
- cache tag compare to hit/miss decision;

- pipeline stall logic to stage-enable register.

Explain how extra load, lower voltage, or a slow corner could consume slack, then name one hardware response and one trade-off it introduces.

---

14. Mastery checkpoint

Close the lesson and answer:

> Why can a logically correct CPU fail when clocked too fast, and how are capacitance, fan-out, power, PVT, and the critical path connected?

A complete answer includes:

- charge stored on gate and wire capacitances;
- finite pull-up/pull-down drive;
- propagation and transition time;
- fan-out increasing effective load;
- dynamic energy scaling approximately with CV^2;
- short-circuit and leakage as additional power;
- PVT changing delay and power;
- the longest constrained register-to-register path bounding clock period;
- timing failure becoming incorrect captured state;
- performance depending on more than frequency alone.

Why? notebook

1. Why does a correct truth table say nothing about maximum clock frequency?
2. Why is a wire part of the circuit rather than an abstract connection?
3. Why can adding buffers reduce delay while increasing power and area?
4. Why does lowering voltage save substantial dynamic power but threaten timing margin?
5. Why does an idle CPU still consume power?
6. Why can glitches consume energy even when the final output is unchanged?
7. Why is “typical at room temperature” insufficient for reliability?
8. Why might the critical path change after placement or at another PVT corner?
9. Why can a higher-frequency CPU deliver less useful performance?
10. How can transistor-level delay eventually surface as Linux thermal throttling or reduced boost residency?

Mental model at the end

```
Logic values are voltages on capacitive nodes.
Drivers move finite charge through finite conducting paths.
Load and fan-out change delay, edge rate, and energy.
Switching power is roughly αCV²f; overlap and leakage add more.
PVT means there is no single universal delay.
The slowest constrained path consumes the clock budget.
Timing and power limits shape CPU performance and reliability.
```

References used selectively

- Holger Vogt et al., *ngspice User's Manual*, Version 47, 11 August 2026, transient analysis, measurement commands, MOSFET devices, and level-1 model: https://ngspice.sourceforge.io/docs/ngspice-html-manual/manual.xhtml
- ngspice release history, confirming release 47 and its 11 August 2026 documentation: https://ngspice.sourceforge.io/news.html
- Ivan Sutherland, Bob Sproull, and David Harris, *Logical Effort: Designing Fast CMOS Circuits*, Morgan Kaufmann, 1999; companion material for the topology/load/staging intuition: https://pages.hmc.edu/harris/research/letalk.pdf
- David Money Harris, “CMOS VLSI Design — Power,” for activity, capacitive switching, short-circuit, and leakage categories: https://pages.hmc.edu/harris/cmosvlsi/4e/lect/lect7.pdf
- Mark Horowitz, “Computing's Energy Problem (and what we can do about it),” ISSCC 2014, for the modern system-level importance of data movement and energy limits: https://doi.org/10.1109/ISSCC.2014.6757323
- David A. Patterson and John L. Hennessy, *Computer Organization and Design*, processor-performance and pipelining chapters, for the connection between stage delay, clock period, CPI, and delivered performance.

Next bridge: timing between combinational stages leads directly to setup, hold, clock skew, latches, flip-flops, and metastability—the rules that let a CPU capture changing signals as reliable state.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Delay Power

Format: Python lab

What you will do: Sweep a small CMOS inverter's load, then sample three illustrative corners.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
python3 04-cpu-and-chip-design/challenges/day-004-delay-power.py
```

- View lab source on GitHub

Day 5 — Sequential Timing and Metastability

Target time: approximately 2–3 hours

- Feedback, latches, flip-flops, and the clock: 40–50 minutes
- Setup, hold, clock-to-Q, skew, and jitter: 50–60 minutes
- Metastability, synchronizers, and CDC warnings: 40–50 minutes
- CPU timing exercises and mastery evidence: 25–35 minutes

> An ALU may produce the right result eventually. What makes it safe for a CPU register to call that result its new state at one particular clock edge?

Why this day exists

Combinational logic has no intentional memory. Change its inputs and, after finite propagation delay, its outputs change. A CPU cannot be built from that alone. Its program counter must retain an address, its register file must retain operands, and a pipeline stage must preserve one instruction's result while the next instruction enters the logic behind it.

Feedback creates the possibility of state. Clocked storage makes state changes orderly. Timing constraints describe when that order is reliable.

```
old register state
    → finite clock-to-Q delay
    → combinational CPU work
    → setup at the next register
    → next edge captures new state
```

That arrow chain is the purpose of today's lesson. It is also where the neat digital model touches analog reality: a storage element asked to decide at the wrong time can become metastable.

By the end, you should be able to:

- explain how controlled feedback preserves state;
- distinguish an SRAM or DRAM bit cell from architectural memory;
- explain qualitatively why registers, caches, and main memory use different storage organizations;
- distinguish a level-sensitive latch from an edge-triggered flip-flop;
- explain what a clock coordinates and what it does not compute;
- reason about setup, hold, clock-to-Q, skew, and jitter;
- explain metastability as analog resolution, not a third logic value;
- state why synchronizers reduce failure probability rather than remove it;
- recognize common clock-domain-crossing mistakes;
- connect timing directly to CPU frequency, pipelining, reset, interrupts, device inputs, and multi-clock SoCs.

1. Feedback turns a present signal into remembered history

A combinational block is described by its current inputs:

```
output = F(current inputs), after propagation delay
```

A sequential block also depends on state:

```
next_state = F(current_state, current inputs)
```

The simplest route to state is feedback—returning an output to an input. Cross-coupled restoring gates can reinforce either of two stable conditions:

```
Q=0, Q_bar=1
Q=1, Q_bar=0
```

Each side helps hold the other side in its opposite state. This is physical memory only while the circuit remains powered; ordinary CPU registers are volatile.

Feedback is not automatically useful. An arbitrary combinational loop may oscillate or settle unpredictably. Storage circuits arrange feedback and control inputs so that stable states can be deliberately selected and held.

Historically, reliable state was one of computing's hardest resources. Relays, vacuum-tube flip-flops, delay lines, electrostatic stores, magnetic cores, and semiconductor memories offered different compromises in speed, density, power, and reliability. The modern register is easy to draw as a box precisely because generations of circuit engineering are hidden inside that box.

CPU path

State divides one continuous electrical machine into understandable steps:

```
PC holds address N
    → instruction and next-PC logic settle
    → PC captures address N+1
    → the next interval begins from a durable value
```

Without storage boundaries, “instruction N” and “instruction N+1” would not have clean architectural moments.

2. Bit cells are physical mechanisms, not architectural memory

Week 1 treated a register and memory as boxes that preserve bits so a minimal CPU could fetch instructions and retain results. We can now unpack those boxes one level without turning this into a memory-circuit design course.

A **bit cell** is a circuit that physically retains one bit under specified electrical conditions. **Architectural memory** is the programmer-visible model: addressed bytes or words, load/store behavior, ordering rules, and faults. Between those layers are register files, caches, controllers, buses, error correction, coherence, address translation, and packaging. An architectural load does not directly select “a transistor”; it initiates a memory-system operation that may encounter several structures.

6T SRAM: feedback stores the state

A common six-transistor SRAM cell contains:

- two cross-coupled CMOS inverters, four transistors total, whose outputs reinforce the stable states `Q=1, Q_bar=0` or `Q=0, Q_bar=1`;
- two access transistors controlled together by a **word line**;
- two complementary **bit lines**, conventionally `BL` and `BL_bar`, shared by many cells in a column.



With the word line inactive, the access transistors isolate the cell and the cross-coupled inverters hold its state while powered. To write, peripheral circuits drive `BL` and `BL_bar` to opposite levels strongly enough that asserting the word line forces the cell into the requested stable state.

For a read, the two bit lines are typically precharged. The word line then connects the selected cell. The side storing zero causes one bit line to discharge slightly relative to the other; a sense amplifier detects that small differential rather than waiting for the cell to drive a long, capacitive column to a full logic level. The read must not overpower and flip the cell. Its cross-coupled feedback retains and restores full internal logic levels after the small read disturbance and after the word line closes.

This is SRAM—**static** RAM—because feedback retains the bit without periodic refresh while power and operating conditions remain valid. “Static” does not mean nonvolatile, zero-power, or zero-latency.

DRAM: charge stores the state

A simplified DRAM cell uses one access transistor and one capacitor. The word line controls the transistor; the bit line connects the capacitor to sensing and write circuitry:



The stored charge is approximately:

```
Q = C V
```

A charged versus less-charged capacitor represents the bit. Because the cell capacitor is tiny, reading shares its charge with a much larger bit line and creates only a small voltage change. A sense amplifier detects and amplifies that change to a full logic level. This sensing disturbs the original charge, so an ordinary DRAM read is internally **destructive** and the sensed value is written back as part of activation/restoration.

Charge also leaks through real devices. An RC picture gives useful intuition: a finite leakage path gradually changes capacitor voltage. DRAM controllers therefore refresh rows before leakage makes their state ambiguous. Real DRAM adds row buffers, banks, activation/precharge commands, timing rules, and refresh scheduling; the one-transistor/one-capacitor cell is only its storage core.

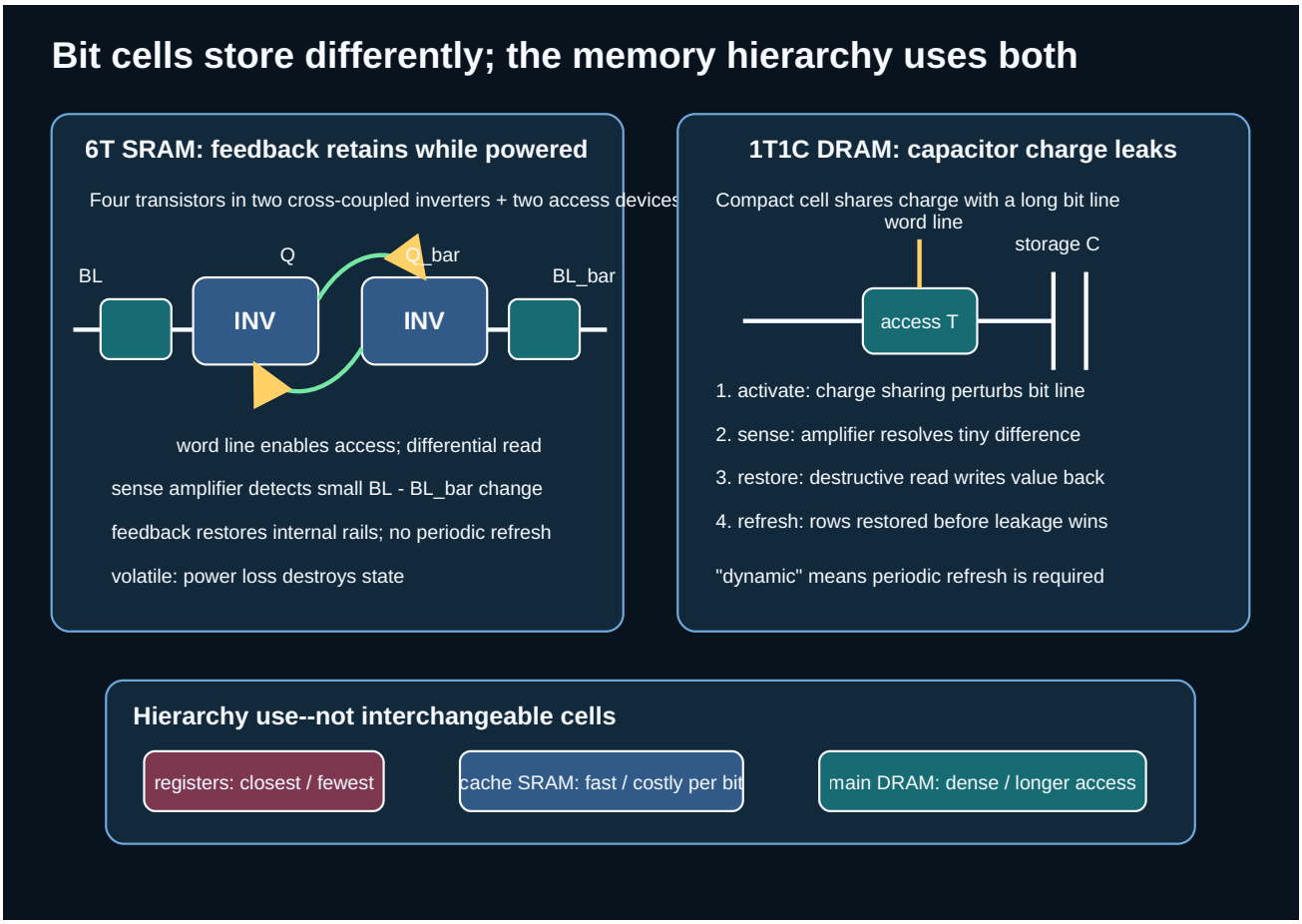
Why the hierarchy uses different storage

No single implementation simultaneously gives minimum latency, many ports, maximum density, and minimum cost per bit:

```
CPU registers   few bits, very close and heavily ported; lowest access latency,
                but expensive in transistors, wiring, clock load, and area
cache SRAM      denser arrays with decoder/sense circuitry; fast enough to
                keep recently used blocks near the core, but costly per bit
main DRAM       much denser 1T1C cells and lower cost per bit; access requires
                array commands, sensing/restoration, and sometimes refresh
```

These are organization-level tendencies, not guarantees for every product. Registers are often built from flip-flops or specialized register-file cells, not ordinary 6T cache SRAM cells. Caches contain SRAM arrays plus tags, comparators, replacement state, and control. Main memory contains far more than isolated DRAM capacitors.

This distinction explains a cache miss physically without confusing layers. The requested architectural address was not found in a nearby cache's SRAM tags/data, so the cache controller obtains a whole block from a lower level, potentially reaching DRAM with its longer command, sensing, and restoration path. The CPU may stall or continue other work, then retry or complete the load after the block is installed. The software-visible operation is still a load; the miss is a microarchitectural event caused by where a copy was—or was not—present.



SRAM retains state with feedback, while DRAM charge requires sensing, restoration, and refresh; each serves a different hierarchy role.

---

3. A latch is open for a level; a flip-flop samples around an edge

Level-sensitive latch

A D latch has a data input, an enable, and an output:

```
enable active    Q follows D after propagation delay
enable inactive  Q holds the last accepted value
```

While enabled, the latch is **transparent**. Changes can pass through it. When enable becomes inactive, feedback preserves the last value.

Latches are legitimate design elements. A disciplined latch-based pipeline can borrow time between phases. The cost is more subtle reasoning about transparency, overlapping phases, races, and time borrowing.

Edge-triggered flip-flop

An edge-triggered D flip-flop accepts data only around an active clock edge:

```
before edge   D must arrive early enough
at edge       the internal decision is made
after edge    Q changes after clock-to-Q delay
between edges later D changes do not immediately change Q
```

Its circuit is not an infinitely fast snapshot device. “At the edge” is a digital abstraction over a small analog sampling aperture.

Why CPU diagrams favor flip-flops

Edge-triggered registers produce a simple pipeline model:

```
launch edge N
    → register output changes
    → combinational path settles
capture edge N+1
```

Real processors also contain SRAM arrays, level-sensitive structures, clock gating, multiple clock domains, and specialized circuits. The edge model is a foundation, not a claim that every state bit uses one identical cell.

---

4. What the clock does—and does not do

The clock provides coordinated sampling boundaries. It lets many state elements distinguish old state from new state.

It does not:

- perform arithmetic;
- push bits through gates;
- guarantee that data arrived in time;
- make an asynchronous input synchronous;
- erase transistor delay or analog behavior.

Between edges, combinational circuitry responds continuously:

```
register outputs
    → decoder / mux / ALU / compare / address logic
    → candidate register inputs
```

At an enabled edge, selected storage elements attempt to capture those candidates. A synchronous design works because every timed path is checked against the clock contract.

This approach became dominant because it converts a global continuous-time problem into repeated local obligations. If all relevant paths meet their constraints under the specified process, voltage, temperature, and clock conditions, designers can reason cycle by cycle.

---

5. Setup timing: can data arrive before the next edge?

Consider two registers separated by combinational logic:

```
launch register ──► logic and routing ──► capture register
       edge N                               edge N+1
```

Important quantities are:

- **clock-to-Q (`t_clk→Q`)** — delay from the source's active clock edge until its output is valid;
- **combinational delay (`t_comb`)** — propagation through logic and routing;
- **setup time (`t_setup`)** — how long the destination input must be stable before its active edge.

For an ideal common clock, a first setup equation is:

```
T_clock ≥ t_clk→Q,max + t_comb,max + t_setup
```

Suppose:

```
t_clk→Q,max = 80 ps
t_comb,max  = 720 ps
t_setup     = 100 ps
```

The period must be at least 900 ps before adding clock uncertainty and other margins. The corresponding ideal upper frequency is about 1.11 GHz.

This explains a central CPU trade-off. If a register-file-to-ALU-to-writeback path is too long, designers can:

- lower frequency;
- simplify or restructure logic;
- improve cells, placement, or routing;
- reduce fan-out;
- insert a pipeline register.

Pipelining can shorten the work per stage and raise clock frequency, but it adds latency and creates hazard, forwarding, stall, and branch-recovery problems. Timing pressure changes architecture.

---

6. Hold timing: can old data remain stable after this edge?

Setup asks about arrival before a future capture. Hold asks whether the old value remains stable briefly after the current capture edge.

```
                 active edge
                      |<-- t_hold -->|
data must remain stable through this interval
```

A simplified same-clock hold condition is:

```
t_clk→Q,min + t_comb,min ≥ t_hold
```

The minimum delays matter because a hold failure is a race: newly launched data reaches the destination too quickly and disturbs the decision associated with the same nominal edge.

Slowing the clock normally does not repair this race. The next edge is not the problem. Physical-design tools may add delay, change cells, or reroute a short path.

Setup and hold form a sampling aperture

```
stable old value   setup window | edge | hold window   stable new value
──────────────────────────────────┼──────────────────────────────────────
```

The input should not change within that aperture. This is evidence that a flip-flop does not decide at a dimensionless instant; internal nodes need time to amplify a small voltage difference into a full logic level.

---

7. Clock skew and jitter spend margin

The clock is distributed through real buffers and wires. It does not arrive at every register simultaneously.

Clock skew is the difference in arrival time between clock endpoints in a cycle. If the capture clock arrives later than the launch clock, a setup path may gain time while a hold path loses margin. The opposite arrival relation can reverse those effects. Tool sign conventions vary, so reason from actual arrival times.

Clock jitter is variation of an edge from its ideal time across cycles. Oscillator and PLL noise, supply variation, and coupling all contribute.

```
ideal:   |----------|----------|----------|
actual:  |---------|------------|---------|
```

Skew is primarily a spatial arrival difference. Jitter is temporal edge variation. Static timing analysis incorporates clock uncertainty and checks paths across operating corners. An untimed RTL waveform cannot establish that silicon meets either setup or hold.

---

8. Metastability is unresolved analog competition

If data changes inside the sampling aperture, the storage element can enter a delicately balanced internal condition. Its output may remain near a switching threshold longer than normal before tiny device imbalance and noise drive it toward a valid rail.



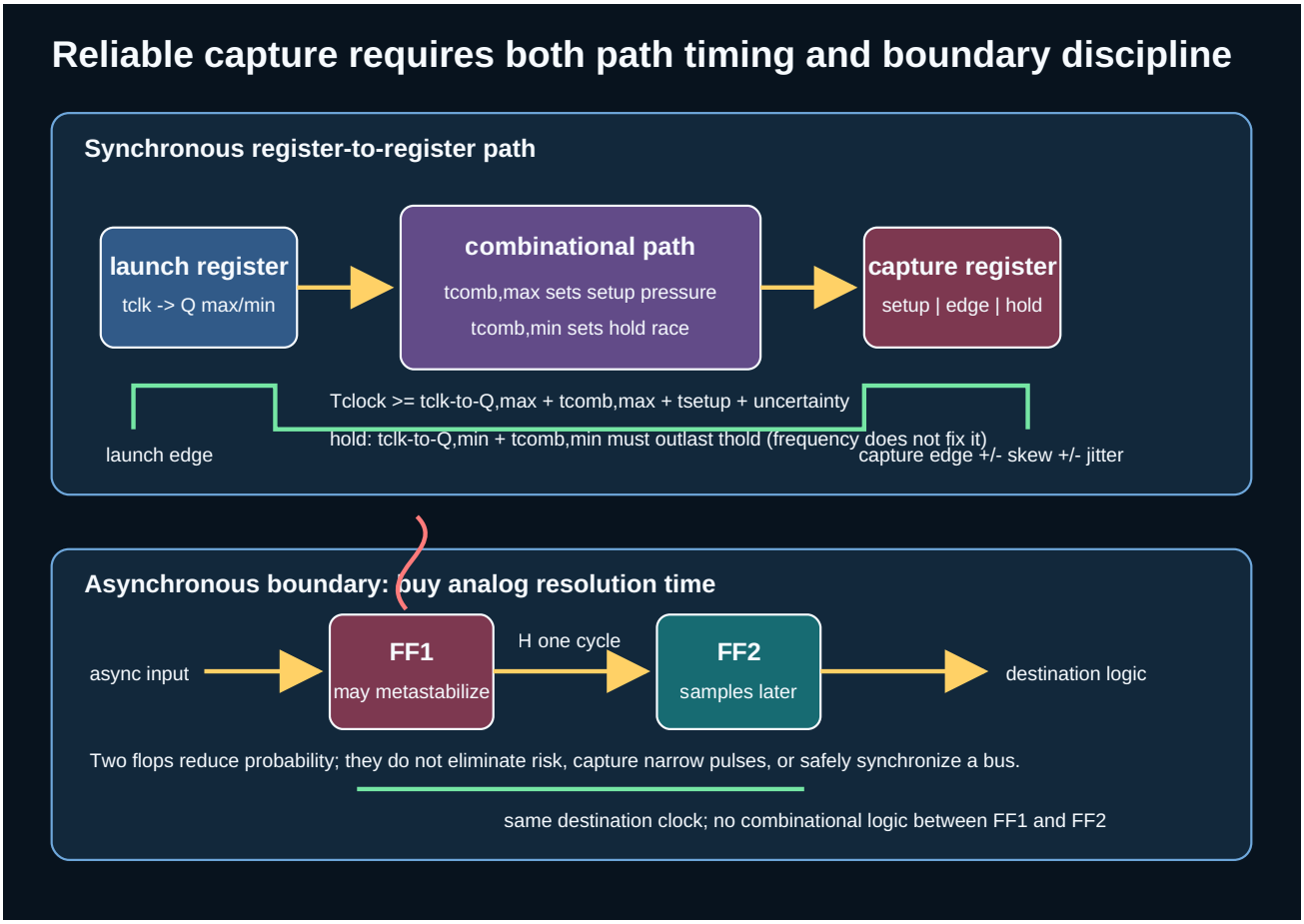
Metastability is:

- an analog condition in a bistable circuit;
- probabilistic in occurrence and resolution duration;
- possible whenever an input has no guaranteed timing relationship to the sampling clock;
- capable of violating timing at downstream logic.

It is not:

- a stable third Boolean value;
- the same thing as a SystemVerilog `X`;
- guaranteed to resolve to the old value or the new value;
- eliminated by simulation, a faster simulator, or a type declaration.

An RTL simulator models discrete values and scheduling rules. It may show `X` when drivers conflict or a value is unknown, but it does not solve the analog differential equations of a metastable storage node. Conversely, a clean zero/one simulation does not prove metastability safety.



Setup, hold, skew, and jitter bound synchronous capture; a two-flop synchronizer buys resolution time without eliminating risk.

Why ordinary synchronous paths avoid it

Static timing analysis tries to prove that register-to-register paths meet setup and hold across specified corners. Asynchronous inputs cannot satisfy that proof merely by declaration because their transitions are unrelated to the destination clock.

Examples relevant to systems programmers include:

- an external interrupt pin entering a CPU clock domain;
- a device status signal crossing into an interconnect clock;
- reset deassertion;
- communication between CPU, memory-controller, PCIe, and peripheral clock domains;
- GPIO or board-level signals.

---

9. A synchronizer buys resolution time

For a slowly changing single-bit level, a common structure is two destination clocked flip-flops in series:

```
async input ──▶ FF1 ──▶ FF2 ──▶ destination logic
                  destination clock
```

FF1 can become metastable. FF2 samples one cycle later, giving FF1 most of a clock period to resolve before its value is consumed. There is still a nonzero probability that resolution takes too long. The synchronizer reduces that probability to an engineered level.

Do not place combinational destination logic between FF1 and FF2. Keep the stages physically suitable and tell CDC/timing tools that this is an intentional synchronizer so the path receives appropriate treatment.

MTBF intuition

A common qualitative model has the shape:

```
MTBF grows approximately exponentially with available resolution time
MTBF falls as destination clock frequency rises
MTBF falls as asynchronous transition rate rises
```

Library characterization supplies device-dependent constants. One extra cycle can improve MTBF enormously because resolution probability has an exponential tail.

MTBF is a statistical population/time measure, not a schedule. It does not say which event fails or promise that this boot is safe. System reliability must account for synchronizer count, product population, operating lifetime, voltage, temperature, and required failure rate.

What two flip-flops do not solve

They do not automatically make these crossings correct:

- **narrow pulses** — the destination may never sample the asserted level;
- **multi-bit buses** — bits can settle on different cycles, producing a word that never existed at the source;
- **counters** — several binary bits may change together;
- **data plus valid without a protocol** — control and payload can lose alignment;
- **reset release** — asynchronous deassertion can violate recovery/removal requirements.

Typical solutions depend on the transfer:

```
single persistent level     two-flop synchronizer
event/pulse                 toggle, pulse-stretch, or handshake
multi-bit payload           handshake with stable data
continuous stream           asynchronous FIFO
cross-domain counter        Gray-coded representation plus synchronization
reset                       asynchronous assert, synchronized deassert
```

These are patterns, not substitutions for CDC analysis. A protocol must state when data becomes stable, when it may change again, and how acknowledgement works.

---

10. Why a kernel or driver engineer should care

CDC is normally implemented below software, but its contract appears in software-visible behavior:

- status bits may be delayed by synchronizer stages;
- MMIO register documentation may require polling or handshakes;
- reset and clock-enable sequences may require bounded waits;
- hardware event counters may use snapshot protocols;
- device FIFOs bridge independently clocked producer and consumer logic;
- an interrupt line is synchronized before internal handling;
- power-management transitions can stop or change clocks.

`volatile`, barriers, locks, and C atomics solve software/compiler/CPU ordering problems under their own contracts. They do not repair a broken hardware clock-domain crossing. Conversely, a correct synchronizer does not replace the memory-ordering rules software needs.

This distinction matters when debugging “impossible” intermittent failures:

```
software race?
memory-ordering bug?
device protocol violation?
reset/clock sequencing error?
hardware CDC defect?
```

The symptoms can look similar while the responsible layer differs.

---

11. Timing exercises

Exercise A — setup budget

A CPU stage has:

```
t_clk→Q,max       70 ps
decode + mux     260 ps
ALU              480 ps
routing          110 ps
t_setup           80 ps
clock uncertainty 60 ps
```

1. Calculate the minimum period under this simplified model.

2. Calculate the corresponding ideal frequency.
3. If a pipeline register splits the logic into 430 ps and 420 ps stages, what new non-logic overhead appears in both stages?
4. Why does doubling the number of stages not automatically double useful performance?

Exercise B — hold race

Use:

```
t_clk→Q,min = 35 ps
t_comb,min  = 10 ps
t_hold      = 55 ps
```

1. Is the simplified hold condition met?
2. Why is lowering frequency not the direct fix?
3. Name one physical change that could add minimum-path delay.

Exercise C — crossing choice

Choose a mechanism and justify it:

1. a board button that remains pressed for milliseconds;
2. a one-source-cycle completion pulse entering a slower domain;
3. a 64-bit DMA descriptor plus a ready indication;
4. a continuous stream between unrelated clocks;
5. a binary counter observed from another domain.

Do not answer every case with “two flip-flops.”

---

12. Mastery evidence

Explain

Without notes:

- explain how feedback permits remembered state;
- distinguish 6T SRAM feedback from DRAM capacitor charge and restoration;
- distinguish either bit cell from architectural memory;
- connect register/cache/DRAM tradeoffs to a cache miss;
- distinguish latch transparency from flip-flop edge sampling;
- explain why the clock coordinates capture but does not perform computation;
- derive the simple setup and hold inequalities;
- distinguish skew from jitter;
- explain metastability without calling it `X` or a third logic level;
- explain what synchronizer MTBF means and does not mean;
- explain why a multi-bit bus needs a protocol.

Draw

Draw:

1. cross-coupled restoring gates with two stable states;
2. a transparent latch interval and an edge-triggered capture;
3. launch register → combinational CPU path → capture register;
4. setup and hold windows around one edge;
5. two-flop single-bit synchronizer;
6. asynchronous FIFO as two clock domains separated by storage and pointer transfer—block-level only.

Observe

There is intentionally no HDL metastability lab today. Preserve:

- your setup and hold calculations;
- a timing diagram annotated with data arrival and clock arrival;
- one CDC classification from a real SoC/device block diagram if available;
- a statement of why a digital waveform cannot prove metastability MTBF.

This absence is part of the lesson: do not manufacture false evidence with an RTL `X`.

CPU checkpoint

Answer:

> How do setup, hold, clock-to-Q, clock distribution, and metastability limit the claim that "all CPU registers update at the clock edge"?

A strong answer separates the architectural abstraction from the electrical contract that makes the abstraction reliable.

Also be able to state why that register abstraction does not imply that cache SRAM, main-memory DRAM, and CPU registers use the same bit cell.

Why? notebook

1. Why does arbitrary feedback oscillate while designed feedback can store?
2. Why can a latch pass more than one transition while transparent?
3. Why is a flip-flop's edge really a sampling aperture?
4. Why does setup use maximum delay while hold uses minimum delay?
5. Why can lowering frequency help setup but not an ordinary same-edge hold violation?
6. Why can useful clock skew help one check and hurt another?
7. Why is metastability inevitable at asynchronous boundaries?
8. Why does another synchronizer stage improve probability rather than guarantee correctness?
9. Why can independently synchronizing every bus bit create an incoherent word?
10. Why can a software barrier not fix a physical CDC error?

References used selectively

- David Money Harris and Sarah L. Harris, *Digital Design and Computer Architecture*, chapters on sequential logic and synchronous timing, for latch/flip-flop behavior and register-to-register timing.
- Neil H. E. Weste and David Harris, *CMOS VLSI Design*, chapters on sequential circuits, clocking, and synchronization, for the physical timing and metastability model.
- Clifford E. Cummings, “Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog,” SNUG Boston 2008, for CDC structures and failure modes: http://www.sunburst-design.com/papers/CummingsSNUG2008Boston_CDC.pdf
- Intel, *Metastability in FPGAs*, for MTBF intuition and synchronizer resolution time: https://www.intel.com/content/www/us/en/docs/programmable/683082/current/metastability.html
- AMD, *Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906)*, sections on clock-domain crossing and timing analysis: https://docs.amd.com/r/en-US/ug906-vivado-design-analysis

Next bridge: now that a register-to-register transfer has a physical timing contract, Day 6 uses one small RTL design to expose how engineers describe that transfer as an `always_ff` register abstraction—not as a 6T SRAM or DRAM cell—simulate its scheduled behavior, and inspect a waveform, then returns immediately to the CPU datapath.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Day 6 — SystemVerilog RTL and Waveforms

Target time: approximately 2–3 hours

- RTL as a hardware abstraction: 35–45 minutes
- One ALU/register stage: 45–55 minutes
- Testbench, event scheduling, and waveform reading: 45–55 minutes
- Run, explain, and reconnect to the CPU: 30–40 minutes

> If SystemVerilog looks like software, why does an `always_ff` block become state while an `always_comb` block becomes gates?

Why this day exists

Drawing every gate becomes impractical long before a CPU becomes interesting. Hardware description languages let engineers specify useful structure and behavior at a higher level, then use different tools for different questions:

```
RTL source
  ├─ simulator: what scheduled behavior does this model produce?
  ├─ linter: what suspicious or inconsistent description is present?
  └─ synthesis: what hardware graph can implement the synthesizable subset?
```

SystemVerilog is not “C for hardware.” Source order is not the hardware's execution order, time can be part of simulation, and many constructs have no synthesizable meaning. RTL—**register-transfer level**—focuses on values transformed by combinational logic and transferred between state elements on clock boundaries.

Today uses one example only: an eight-bit ALU feeding an output register. The point is to reveal the boundaries among:

- combinational intent;
- clocked state;
- simulation scheduling;
- a testbench;
- a waveform;
- synthesizable hardware;
- the later CPU datapath.

This is not a broad SystemVerilog language tutorial and not an advanced verification course.

By the end, you should be able to:

- read a small `module` and its typed ports;
- distinguish `always_comb` from `always_ff`;
- explain blocking (`=`) and nonblocking (`<=`) assignments in this design;
- explain why the testbench is executable simulation support, not chip logic;

- read the result before and after a clock edge in a VCD waveform;
- describe the relevant simulator event ordering without memorizing the full language scheduler;
- identify which parts are intended for synthesis;
- map the example back to one stage of a CPU datapath.

---

1. RTL describes relationships in hardware

In software, one thread normally executes statements in sequence. In hardware, an adder, an AND network, and a comparator can all exist and react at once. RTL source is a notation for such concurrent structure and for clocked updates.

Our design boundary is:

```
a_in ─┐
      ├──► selected ALU operation ──► result register ──► result_out
b_in ─┘                ▲                      ▲
                     op_in                  clock

valid_in ──────────────────────────► valid register ──────► valid_out
```

The combinational ALU continuously derives a candidate value from `a_in`, `b_in`, and `op_in`. The register captures that candidate at a rising edge.

This resembles a tiny CPU execution stage:

```
operand registers / bypass muxes
    → ALU and selection logic
    → pipeline register
    → next stage
```

The example omits register files, hazards, forwarding, flags, exceptions, and control. Those return in the CPU lessons. For now, one boundary is enough.

---

2. Read the module interface as wires crossing a boundary

The lab module begins conceptually as:

```
module alu_register_stage (
    input  logic       clk,
    input  logic       rst,
    input  logic       valid_in,
    input  logic [7:0] a_in,
    input  logic [7:0] b_in,
    input  logic [1:0] op_in,
    output logic       valid_out,
    output logic [7:0] result_out
);
```

A module is a design boundary. Ports describe signals entering and leaving that boundary.

`logic [7:0]` is an eight-bit packed vector with indices 7 down to 0. SystemVerilog's four-state simulation domain includes `0`, `1`, `X` (unknown), and `Z` (high impedance). That does not mean a physical wire has a stable voltage named `X`. Four-state values help a simulator expose missing initialization, conflicting drivers, or intentionally unknown behavior.

Signal width is part of the design. The eight-bit addition naturally keeps the low eight result bits here:

```
0xff + 0x01 → 0x00
```

The carry is discarded because this interface has no carry output. A real ISA must define whether it ignores carry, records flags, traps, or uses a wider result. RTL implements a chosen contract; it does not invent one.

---

3. `always_comb`: describe the candidate value

The lab contains:

```
always_comb begin
    case (op_in)
        2'b00:   alu_result = a_in + b_in;
        2'b01:   alu_result = a_in & b_in;
        2'b10:   alu_result = a_in ^ b_in;
        default: alu_result = (a_in < b_in) ? 8'd1 : 8'd0;
    endcase
end
```

`always_comb` states combinational intent. The simulator reevaluates the process when a read input changes, and tools check stronger rules than they would for a generic procedural block.

Every possible `op_in` value assigns `alu_result`. That matters. If one path left it unassigned, preserving the previous value would require storage—an accidental latch in what was meant to be combinational logic.

Blocking assignment here

The block uses blocking assignment:

```
alu_result = expression;
```

Within a procedural activation, the left side updates immediately before the next statement in that process. For straightforward combinational logic, blocking assignment makes intermediate calculations follow ordinary data-dependency order.

Do not turn “blocking for combinational” into a magical syntax rule. The important question is whether the whole process describes a complete combinational function with no remembered value and no conflicting drivers. The convention supports that intent and avoids common simulation races.

What hardware might result?

The source names operations, not final gates. Synthesis may infer:

- an eight-bit adder;
- bitwise AND and XOR logic;
- an unsigned comparator;
- selection logic controlled by `op_in`.

Optimization can restructure or share logic. The source does not specify physical placement, exact cell count, route delay, or maximum frequency.

---

4. `always_ff`: describe the state boundary

The clocked part is:

```
always_ff @(posedge clk) begin
    if (rst) begin
        valid_out  <= 1'b0;
        result_out <= 8'd0;
    end else begin
        valid_out  <= valid_in;
        result_out <= alu_result;
    end
end
```

`always_ff` states that these variables represent flip-flop-like state updated on the rising edge. Reset is synchronous in this example: `rst` is observed at a rising edge. Merely changing `rst` between edges does not immediately clear the outputs.

This is an RTL **register abstraction**: it specifies when state is captured and how the next value is chosen. It does not specify a transistor-level storage circuit. For this small stage, synthesis will normally infer a bank of flip-flops; the source is not a drawing of a 6T SRAM cell, a DRAM cell, or an architectural register file. Day 5's bit cells answer “how can one array retain charge or feedback state?” while `always_ff` answers “what clocked state behavior does this RTL require?”

That is a design choice, not the universal meaning of reset. CPUs and SoCs use synchronous resets, asynchronous resets, reset synchronizers, retention, and power-domain sequencing according to implementation needs.

Nonblocking assignment and simultaneous-looking capture

The clocked block uses nonblocking assignment:

```
result_out <= alu_result;
```

At a rising edge, right-hand sides are evaluated using the current values, then left-hand updates are scheduled for the nonblocking-assignment update region. This models multiple registers sampling old-state-derived values at the same edge.

For example:

```
q1 <= d;
q2 <= q1;
```

At one edge, `q1` receives the old `d`, while `q2` receives the old `q1`. The source order does not turn that into two cycles of data movement within one physical edge.

This is why nonblocking assignment is the standard choice for clocked state. Using blocking assignments carelessly in interacting clocked processes can make simulation depend on process execution order, creating behavior that does not represent the intended parallel capture.

---

5. Current value, candidate value, and captured value

Suppose before a rising edge:

```
a_in       = 3
b_in       = 5
op_in      = ADD
alu_result = 8       combinational candidate
result_out = old     stored output from an earlier edge
```

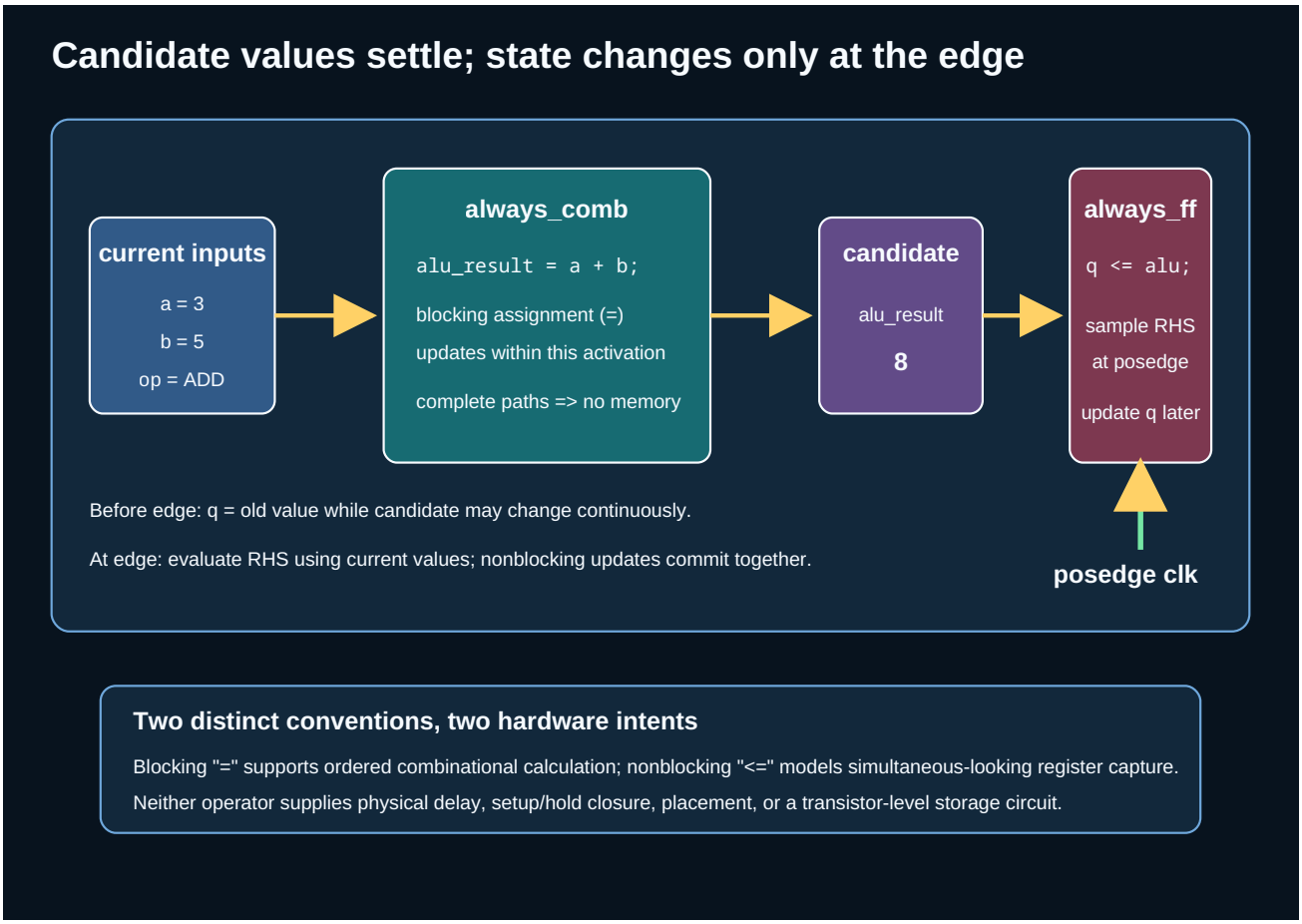
At the rising edge, the `always_ff` process samples `alu_result`. After the nonblocking update executes:

```
result_out = 8
```

The distinction is the whole register-transfer model:

```
between edges: candidate values settle
at edge:       selected candidates are sampled
after edge:    registered outputs update
```

This model assumes the Day 5 physical timing contract is satisfied. RTL simulation does not automatically account for clock-to-Q, setup, hold, routing, skew, or jitter. Those require timing constraints, implementation information, static timing analysis, and sometimes timing-aware simulation.



Blocking assignment computes a complete combinational candidate; nonblocking assignment schedules clocked state capture.

---

6. The testbench is outside the synthesized design

The testbench instantiates the design under test:

```
alu_register_stage dut (...);
```

It then provides an artificial environment:

- an `initial` process sets starting values;
- `forever #5` creates a 10 ns clock;
- a task drives operands and an operation;
- assertions compare observed and expected values;
- `$dumpfile` and `$dumpvars` record a VCD waveform;
- `$display`, `$fatal`, and `$finish` report and control simulation.

These features are valuable precisely because a simulator executes them. They are not intended to become gates in this lab.

A basic assertion

After a rising edge and a small observation delay, the testbench checks:

```
assert (result_out === expected)
    else $fatal(1, "...");
```

This is an immediate assertion: evaluate the condition now and fail if it is not true. The case-equality operator `===` treats `X` and `Z` explicitly, so an unknown result cannot accidentally pass as an ordinary Boolean match.

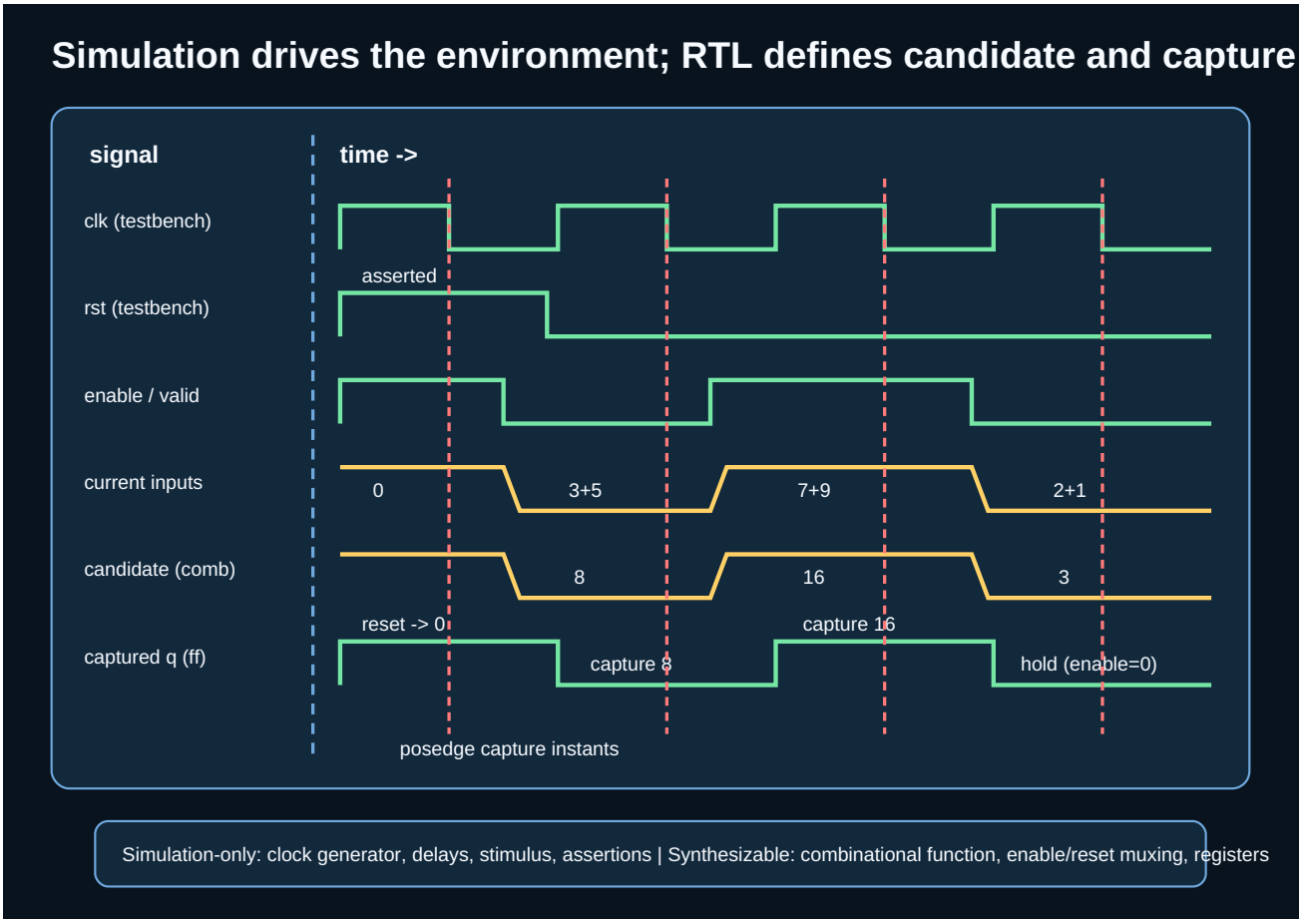
This small check is enough for today's purpose. Industrial verification adds constrained random stimulus, temporal assertions, coverage, formal methods, reference models, and much more. None is needed to understand this boundary.

Why drive at the falling edge?

The testbench changes inputs at a falling edge, halfway before the rising capture edge:

```
falling edge: testbench drives inputs
    ↓ combinational process reevaluates
rising edge: register captures result
    ↓ nonblocking updates occur
testbench observes after #1
```

This avoids a testbench/design race at the same rising edge and makes the waveform easy to read. It is not a physical setup-time proof; `#1` and `#5` are simulation delays in the testbench.



The testbench drives time and stimulus; synthesizable RTL computes candidates and captures enabled values at rising edges.

---

7. The event model: why “at the same time” needs ordering

A simulator cannot literally run every process at once. It uses an event queue. Several events can occur at the same simulation timestamp in ordered regions and repeated zero-time iterations called delta cycles.

For this lab, retain this reduced model:

1. A testbench input change schedules affected combinational logic.
2. `always_comb` computes `alu_result` without advancing simulation time.
3. A rising `clk` schedules the `always_ff` process.
4. The clocked process evaluates right-hand sides.
5. Nonblocking assignments update registered outputs later in the same timestamp.
6. The testbench's `#1` check observes after those updates.

The full IEEE scheduler contains more regions for assertions, program blocks, and other semantics. The reduced model is sufficient to explain this waveform. Learn more scheduler detail when a real race or verification construct requires it.

Delta cycles are not CPU cycles

A delta cycle is zero simulation time used to settle event dependencies. It is not:

- a clock period;
- a gate delay;
- an instruction cycle;
- evidence that silicon performs statements sequentially.

If `alu_result` changes at the same timestamp as an input, that means the RTL model reevaluated in zero modeled physical time. The gate implementation will have propagation delay.

---

8. Run the one lab

Files:

```
04-cpu-and-chip-design/challenges/day-006-alu-register-stage.sv
04-cpu-and-chip-design/challenges/day-006-alu-register-stage-tb.sv
04-cpu-and-chip-design/challenges/day-006-run-alu-register-stage.sh
```

The script:

1. runs Verilator lint over the design and testbench;
2. compiles the same sources with Icarus Verilog;
3. runs the simulation with `vvp`;
4. leaves a VCD file beside the lab;
5. prints an optional GTKWave command but never requires a GUI.

From the repository root:

```
bash 04-cpu-and-chip-design/challenges/day-006-run-alu-register-stage.sh
```

Expected final line from the testbench:

```
PASS: 6 checks; VCD written
```

Predict before running

Write down:

1. When inputs become `3`, `5`, and ADD, when may `alu_result` change?
2. When may `result_out` change?
3. What eight-bit result follows `0xff + 1`?
4. When `valid_in=0`, does this design hold the old result or merely mark the newly captured result invalid?
5. Does a passing simulation establish setup/hold timing?

The fourth question exposes a real interface decision. This stage captures `alu_result` every non-reset edge; `valid_out` says whether consumers may interpret it. Invalid payload bits are don't-care at the protocol level even though the waveform still shows a value.

---

9. Read the VCD without requiring a GUI

The Value Change Dump file is:

```
04-cpu-and-chip-design/challenges/day-006-alu-register-stage.vcd
```

If GTKWave is installed and a graphical session is available:

```
gtkwave 04-cpu-and-chip-design/challenges/day-006-alu-register-stage.vcd
```

Add at least:

```
clk
rst
valid_in
a_in
b_in
op_in
alu_result
valid_out
result_out
```

For one ADD transaction, annotate:

1. the falling edge where the testbench drives inputs;
2. the delta-cycle change of combinational `alu_result`;
3. the next rising edge;
4. the registered-output update after that edge;
5. the testbench check one nanosecond later.

GTKWave is only a viewer. Simulation already completed and assertions already checked behavior. The GUI does not make the evidence more correct.

If no GUI is available, keep the simulation log and inspect the VCD as text:

```
less 04-cpu-and-chip-design/challenges/day-006-alu-register-stage.vcd
```

VCD uses compact identifier codes, so text inspection is less convenient, but it confirms that the artifact is portable trace data rather than an image.

---

10. Simulation is not synthesis

The design module is written in a conventional synthesizable subset:

- fixed-width ports and internal signals;
- combinational arithmetic, comparison, bitwise operations, and `case`;
- one edge-triggered process;
- synchronous reset;
- no testbench delays or file-driven behavior.

The testbench deliberately uses nonsynthesizable simulation features:

- `#` delays;
- an endless clock generator;
- tasks used as stimulus;
- `$dumpfile`, `$dumpvars`, `$display`, `$fatal`, and `$finish`.

Some constructs can be supported differently across synthesis tools, targets, and flows. “SystemVerilog supports it” does not imply “this synthesizer maps it as intended.” The target and tool documentation remain part of the contract.

What each result proves

A passing simulation proves:

> For the tested stimulus, this simulator's execution of this RTL model satisfied these assertions.

Warning-clean lint proves:

> These lint rules found no reported issue in this source under these options.

Neither proves:

- exhaustive functional correctness;
- setup or hold closure;
- metastability safety;
- a particular gate netlist;
- final area, frequency, or power;

- correct CPU behavior around this stage.

Synthesis will be the next representation boundary. Physical implementation comes later still.

---

11. Return to the CPU

Replace the lab names with CPU names:

```
a_in, b_in       selected source operands
op_in            decoded ALU control
alu_result       execute-stage candidate
result_out       pipeline-register value
valid_out        whether the stage carries a real instruction/result
```

Now the missing machinery becomes visible:

- Where do source operands come from?
- How does instruction decode choose `op_in`?
- Which destination register number travels with the result?
- How are overflow, comparison, and exception metadata carried?
- What happens when an older instruction stalls?
- How is a branch redirect handled?
- How does forwarding select a value not yet written back?

Those are CPU architecture questions, not reasons to turn this lesson into a larger HDL tutorial. SystemVerilog is the notation we will use to make the answers executable and inspectable.

---

12. Mastery evidence

Explain

Without notes:

- explain why RTL is not sequential software;
- identify the combinational and stateful parts of the lab;
- explain why every `case` path assigns `alu_result`;
- explain blocking assignment in the combinational process;
- explain nonblocking assignment in the clocked process;
- explain why the testbench waits until after the edge to check;
- distinguish a delta cycle from a clock cycle;
- separate simulation evidence from synthesis and timing evidence.

Draw

Draw one diagram:

```
inputs → four ALU candidates / selection → D input of result register
                                                ↑ rising-edge clock
                                                ↓
                                          registered result
```

Beside it, draw a waveform for one ADD transaction. Mark:

- testbench drive;
- combinational candidate change;
- capture edge;
- nonblocking registered update;
- assertion check.

Observe

Preserve:

- Verilator's warning-clean lint result;
- Icarus's warning-clean compile result;
- the `PASS: 6 checks` message;
- the generated VCD;
- one annotated transaction;
- one sentence saying what the waveform does not prove.

Build—one small change only

Add an enable behavior to a copy of the lab:

```
valid_in=1   capture ALU result
valid_in=0   preserve result_out and clear valid_out
```

Before editing, predict which waveform changes during the invalid transaction. Keep the assignment style and rerun the same lint/simulation loop. This single extension is enough; the next lesson returns to synthesis and CPU structure.

Mastery checkpoint

Answer:

> Starting from stable ALU inputs before a rising edge, explain what the combinational RTL, simulator scheduler, nonblocking assignment, waveform, and intended register hardware each contribute to the observed result.

Do not say that the simulator “executes the ALU and then executes the register” as though those were software statements. The hardware blocks coexist; the simulator schedules a model of their behavior.

Why? notebook

1. Why does incomplete combinational assignment imply remembered state?

2. Why can all ALU operations exist in hardware even though `case` selects one result?
3. Why do clocked assignments observe old values at one edge?
4. Why can a waveform show zero-time combinational changes when gates have delay?
5. Why is a testbench clock generator not synthesized into the design?
6. Why can invalid payload bits still change?
7. Why does passing RTL simulation not prove setup/hold timing?
8. Why can synthesis legally produce a structure unlike the source layout?
9. Why is `X` useful in simulation but not a stable hardware logic level?
10. Where would destination register identity and exception state travel in a real CPU pipeline?

References used selectively

- IEEE Std 1800-2023, *SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, for `always_comb`, `always_ff`, assignment, assertion, and scheduling semantics: https://standards.ieee.org/ieee/1800/7743/
- Icarus Verilog documentation, for SystemVerilog compilation and `vvp` simulation: https://steveicarus.github.io/iverilog/
- Verilator guide, especially lint and timing options: https://verilator.org/guide/latest/
- GTKWave documentation and source, for VCD waveform inspection: https://gtkwave.github.io/gtkwave/
- David Money Harris and Sarah L. Harris, *Digital Design and Computer Architecture*, HDL and sequential-logic chapters, for the connection from RTL to datapaths and state.

Next bridge: synthesis will elaborate this module, infer arithmetic, selection, and flip-flop structures, then optimize a netlist. After that brief tool boundary, the course returns to the CPU's register file, decoder, program counter, control, and instruction execution.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Run Alu Register Stage

Format: shell lab

What you will do: set -euo pipefail

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 04-cpu-and-chip-design/challenges/day-006-run-alu-register-stage.sh
```

- View lab source on GitHub

Day 7 — Synthesis, Netlists, and Physical-Design Overview

Target time: approximately 2½–3 hours

- Simulation versus synthesis: 25 minutes
- RTL to generic and mapped netlists: 50 minutes
- Timing, area, power, and implementation boundaries: 35 minutes
- One Yosys observation and end-to-end trace: 60 minutes

> A simulator showed the register changing after a clock edge. What turns that behavior into hardware structure—and how does it reconnect to the instructions and software that motivated this module?

Synthesis is the bridge between RTL and an implementation netlist. It is not the end of chip design, and this lesson is not a detour into becoming an ASIC physical designer. We need just enough depth to understand what CPU RTL means, why frequency and power constrain microarchitecture, and what lies beneath the ISA boundary used by systems software.

```
software operation
  → ISA instruction
  → datapath/control action
  → RTL state transition
  → inferred mux/adder/register
  → gates and physical switching
```

---

1. Simulation and synthesis are different experiments

A **simulator** executes a model over simulated time. It evaluates events, four-state values (`0`, `1`, `X`, `Z`), clock edges, and testbench stimulus. It asks:

> For these inputs, what values and events does this model produce?

A **synthesizer** interprets the synthesizable subset as a hardware description, then builds and transforms a graph of logic and state. It asks:

> What supported hardware structure implements this behavior?

SystemVerilog can describe activities that are valuable in a testbench but are not circuits:

```
initial begin
    #10 request = 1'b1;
    $display("request changed");
end
```

The delay and print are simulator actions. In contrast, this edge-triggered assignment expresses intended state:

```
always_ff @(posedge clk)
    result <= next_result;
```

“Simulation passed” does not prove synthesizability, timing closure, or correct physical behavior. “Synthesis succeeded” does not prove the design meets its specification. They provide different evidence.

Evidence hands off; it does not accumulate into a silicon proof

The Week 4 tools answer different bounded questions:

```
ngspice circuit experiment   analog voltage/current behavior of the supplied
                             circuit and device models
HDL simulation              scheduled RTL behavior for exercised stimulus
synthesis reports/netlists  inferred and transformed implementation structure
```

The ngspice observations from earlier days support claims about behavior within their educational transistor models; they do not characterize a fabricated process. Day 6's HDL waveform supports a behavioral claim, not a claim about analog delay or metastability. Today's synthesis artifacts support a structural claim, not final silicon timing, placement, routing, layout, power, or manufacturability. Passing all three kinds of experiment does not erase those boundaries.

Four-state values are a model

`X` can reveal uninitialized or conflicting state and `Z` can represent high-impedance behavior in simulation. They are not extra stable Boolean voltage rails inside an ordinary CMOS gate. A synthesis tool may use some unknown-valued cases as don't-care freedom. Never read an RTL `X` as a transistor-accurate analog result.

---

2. The RTL-to-netlist path

The useful conceptual pipeline is:

```
RTL source
  → elaboration
  → generic inference
  → optimization
  → technology mapping
  → target netlist
```

Elaboration makes the requested design concrete

Elaboration chooses the top module, resolves parameters and widths, expands generate constructs, creates instances, connects ports, and turns procedural blocks into an internal representation. A parameterized eight-bit adder and its 32-bit instance are not the same hardware size.

Hierarchy helps humans organize a design, but later optimization can flatten or rearrange it. Source module boundaries need not survive unchanged in the netlist.

Generic inference recognizes hardware intent

Consider:

```
logic [7:0] selected;

assign selected = choose_b ? b : a;

always_ff @(posedge clk) begin
    if (reset)
        result <= '0;
    else if (enable)
        result <= selected + 8'd3;
end
```

A generic representation can contain:

- a mux for `choose_b ? b : a`;
- an eight-bit addition;
- selection for the enable behavior;
- reset behavior;
- eight edge-triggered storage bits.

These are inferred operations, not a promise of particular transistors.

Incomplete combinational assignment can accidentally request storage:

```
always_comb begin
    if (load)
        out = new_value;
    // What drives out when load is zero?
end
```

If the required behavior is “retain the previous value,” a latch is state, not combinational logic. Complete assignments make the intended mux inputs explicit.

Optimization may erase the source-shaped structure

Tools propagate constants, remove unused logic, simplify Boolean expressions, reduce widths, combine muxes, and rewrite arithmetic. `x + 0` can lose its adder. An unobservable register can disappear. Equivalent netlists can look very different.

Therefore, “one `+` in the source means one final cell named ADD” is false. The requirement is preserved behavior under the tool's assumptions, not preserved spelling.

Technology mapping needs a target

Generic mux, adder, and register operations must eventually map into available resources:

- an ASIC library offers characterized standard cells with particular logic functions, drive strengths, timing, area, and power models;
- an FPGA offers fixed LUTs, flip-flops, carry chains, memories, DSP blocks, clocks, and programmable routing.

The lab maps into Yosys's generic/internal primitives. That is enough to observe structural transformation. It is not a foundry-cell result, FPGA bitstream, transistor layout, or final area/power/timing prediction.

---

3. Why CPU designers care about constraints

A CPU is not useful merely because its state transitions are logically correct. Those transitions must happen within electrical and timing limits.

For a simplified register-to-register setup path:

```
launch register
  -- clock-to-Q --> combinational datapath
  -- propagation --> capture register setup window
```

A rough condition is:

```
clock period ≥ clock-to-Q(max) + logic delay(max) + setup + uncertainty
```

Hold analysis checks the earliest arriving change near the same capture edge. Real static timing analysis accounts for characterized arcs, clocks, constraints, modes, corners, and eventually interconnect parasitics.

Constraints describe questions such as:

- What is the clock period?
- When may external inputs arrive?
- When must outputs be valid?
- What load or drive assumptions apply?
- Are any paths intentionally multicycle or logically false?

A clock constraint does not build an oscillator or route a clock wire. Missing or incorrect constraints can produce a clean report for the wrong question.

Area, power, and timing shape microarchitecture

The three goals compete:

- stronger or duplicated logic can improve timing but consume area and power;
- shared hardware can save area but add muxing and delay;
- another pipeline stage can shorten combinational paths and raise frequency, but adds registers, clock load, latency, and hazard/control complexity;
- wider issue, larger caches, more ports, and aggressive speculation can improve some workloads while increasing switching, wiring, and design cost.

This is why pipeline depth, ALU organization, register-file ports, bypass networks, cache size, and clock frequency are not purely abstract choices. The physical costs rise back into CPU architecture.

---

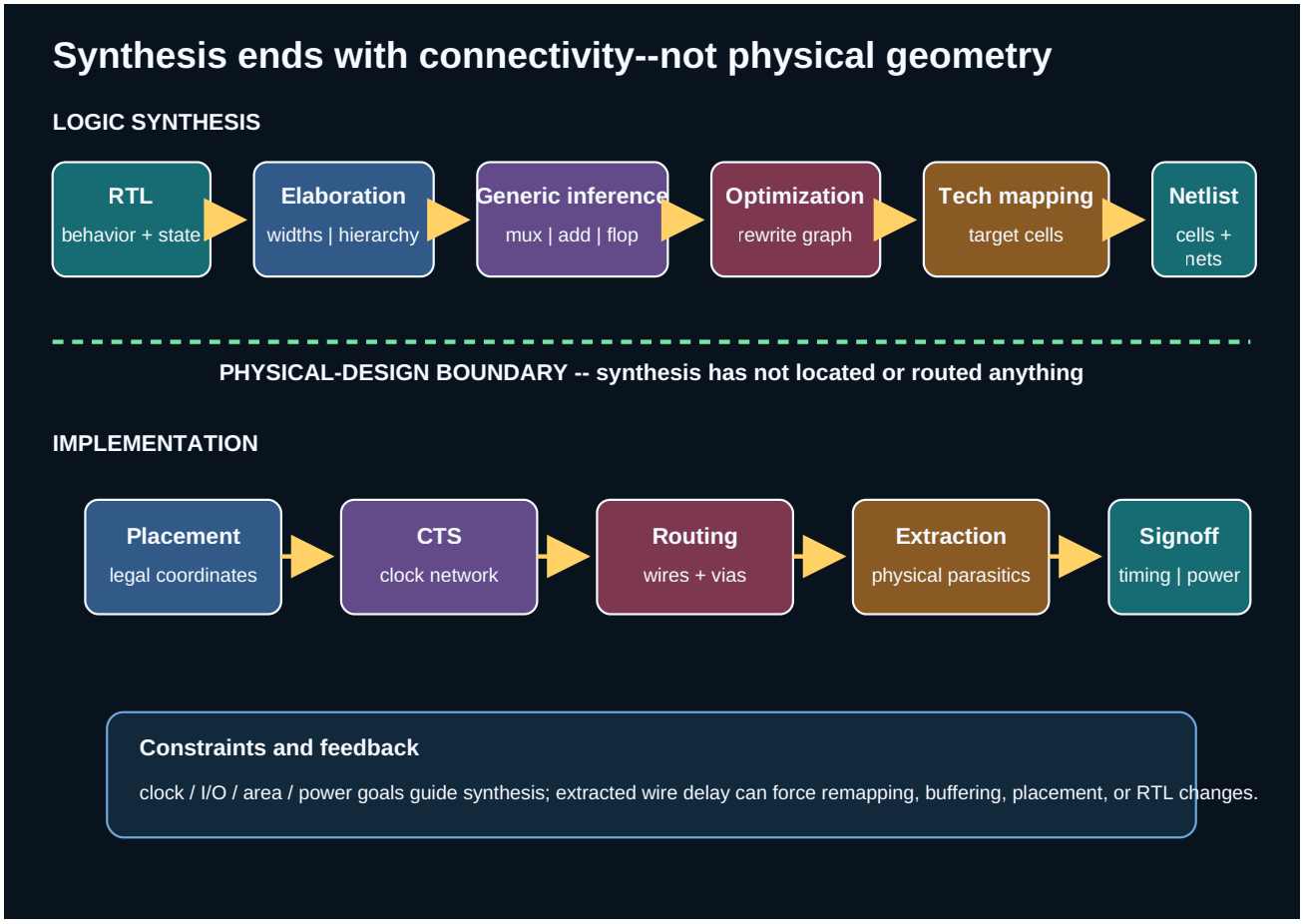
4. Synthesis does not create placement or routing

A **netlist** names cells/operations and their connections. It does not normally say where each standard cell physically sits or exactly which metal segments connect it.

After ASIC logic synthesis, a much larger flow conceptually includes:

```
floorplan → place → clock-tree synthesis → route
          → extract parasitics → timing/power/physical signoff
```

- **Floorplanning** allocates major regions, macros, I/O, and power strategy.
- **Placement** chooses legal cell locations.
- **Clock-tree synthesis (CTS)** builds clock distribution.
- **Routing** assigns physical wires and vias.
- **Signoff** checks timing and physical/electrical requirements using much richer models.



A mapped netlist still lacks placement, clock distribution, routed wires, extracted parasitics, and signoff.

This concise map is enough for our boundary: synthesis has not performed these steps. FPGA implementation likewise still packs, places, and routes the mapped resources before producing configuration data, but it targets a prefabricated programmable fabric rather than custom standard-cell geometry.

---

5. One small Yosys observation

The lab intentionally asks one bounded question:

> Can we see a selected addition and a clocked result change from RTL constructs into generic then mapped structural representations?

From the repository root:

```
bash 04-cpu-and-chip-design/challenges/day-007-yosys-synthesis-lab.sh
```

The script requires Yosys and was validated with Yosys 0.67. If `iverilog` and `vvp` are available, it first runs a small behavioral check. It writes all generated logs, statistics, Verilog/JSON netlists, and DOT graphs under `/tmp`, never into tracked source directories.

Before running, predict:

1. Which expression implies a mux?
2. Where should an addition operation appear?
3. How many bits of state does `result` require?
4. What could optimization remove or reshape?
5. Why might generic and mapped cell counts differ?

After running, inspect:

```
less /tmp/week4-day007-yosys/generic-stat.txt
less /tmp/week4-day007-yosys/mapped-stat.txt
less /tmp/week4-day007-yosys/generic-netlist.v
less /tmp/week4-day007-yosys/mapped-netlist.v
```

The script prints the exact artifact path and key cell names. DOT is a graph description; when Graphviz is installed, the script also renders SVG.

What the observation proves

It provides evidence that Yosys:

- parsed and elaborated this top module;
- recognized process/operation structure;
- emitted generic and mapped netlist representations;
- reported structural statistics.

Separately, when Icarus is available, the RTL testbench provides behavioral evidence for its few exercised cases. This lab does not perform formal or sequential equivalence checking between the RTL and generated netlists.

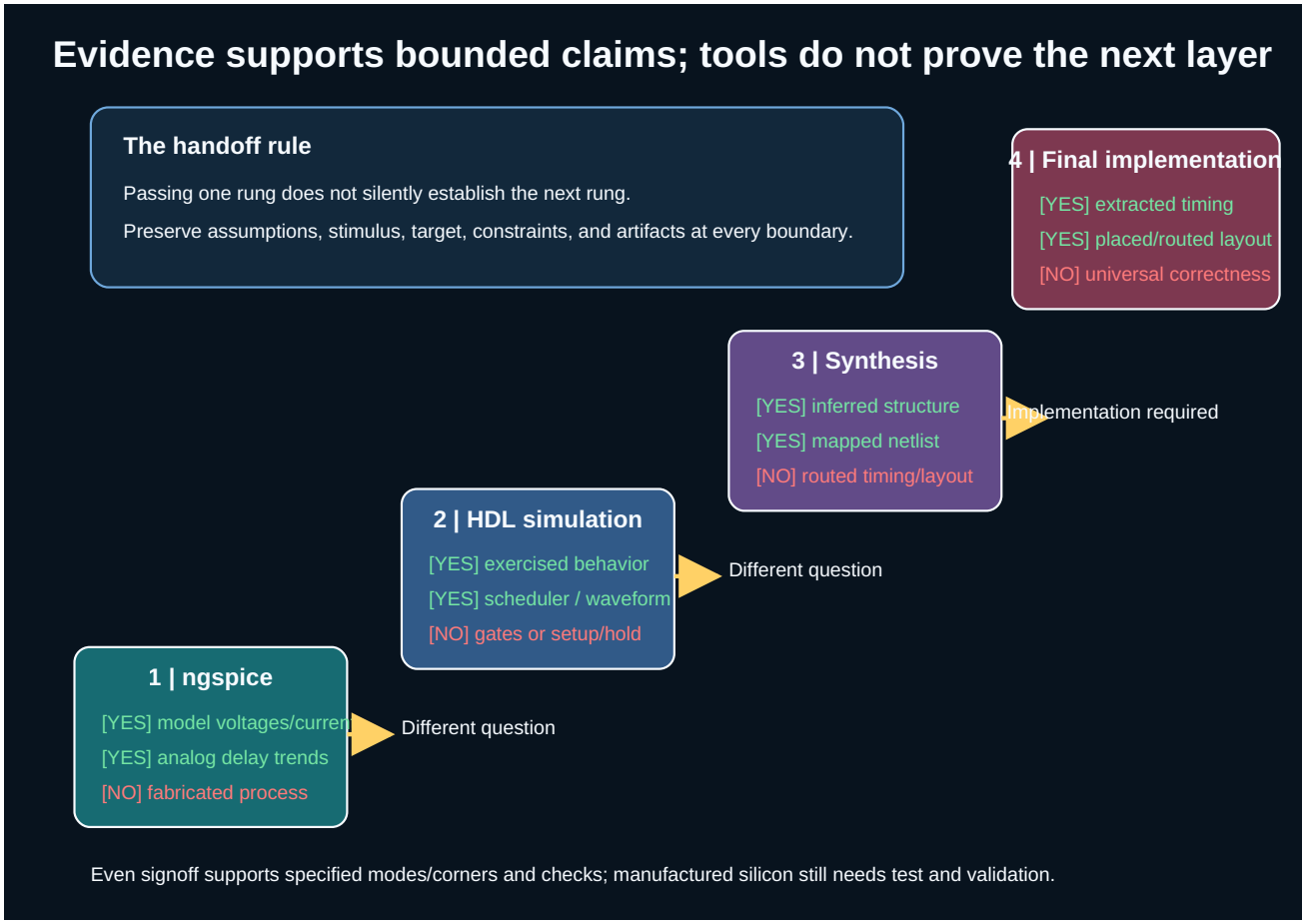
What it does not prove

It does **not** establish:

- transistor-accurate behavior;
- mapping to a real ASIC library or a named FPGA;

- final silicon area, frequency, or power;
- correct placement, CTS, or routing;
- timing closure across operating corners;
- functional correctness beyond the tested cases;
- manufacturability or signoff.

That limitation is part of the lesson, not a weakness to hide.



ngspice, HDL simulation, synthesis, and final implementation answer different bounded questions; no rung silently proves the next.

---

6. Reconnect the netlist to software

Suppose C contains:

```
acc = acc + input;
```

The end-to-end path is not “C becomes transistors” in one jump:

```
C expression
  → compiler-selected ISA instructions
  → fetched instruction bits
  → decoder/control signals
  → register-file reads and operand selection
  → ALU addition
  → result captured in architectural state
  → later instructions observe that state
```

Inside the implementation, operand selection can involve muxes, addition can involve carry logic or a target-specific arithmetic structure, and writeback ends at clocked state. Each gate transition is ultimately charge moving through transistor networks. RTL and synthesis let engineers manage that enormous composition without pretending the lower physical layer vanished.

The ISA remains the contract seen by the compiler, operating system, and application. Microarchitecture decides how a CPU realizes that contract: single-cycle, multicycle, pipelined, cached, speculative, or otherwise.

The bridge back to systems

The next questions return upward:

- How do pipelining, hazards, forwarding, and stalls preserve ISA behavior?
- How do caches and memory interfaces change latency without changing the architectural memory contract?
- How do the MMU, page tables, TLB, privilege levels, exceptions, and interrupts connect CPU mechanisms to an OS?
- How does a kernel establish execution state, handle traps, and isolate processes?

Those questions lead toward CPU microarchitecture and the guided ToyOS work. The HDL/chip material supplies necessary causal depth; it is not a divergent specialization into professional ASIC implementation.

---

7. Mastery: Explain · Draw · Observe · Build

Explain

Without notes:

- distinguish simulation, elaboration, synthesis, optimization, and mapping;
- explain how mux, adder, and flip-flop behavior can be inferred;
- explain why a netlist is not placement or routing;
- connect timing/area/power pressure to one CPU design choice;
- trace one software addition through ISA, datapath, RTL, netlist, gates, and transistor switching.

Draw

Draw one diagram:

```
software → ISA → decoder/control → datapath/register
         → RTL → generic netlist → mapped target resources
         → physical implementation
```

Add a return arrow showing that physical delay, area, and power constrain pipeline, cache, and datapath choices.

Observe

Preserve:

- `yosys -V`;
- the optional Icarus simulation result;
- generic and mapped statistics;
- one generic cell and its mapped structural replacement;
- a three-line handoff stating what ngspice, HDL simulation, and synthesis each establish;
- one sentence for each conclusion the artifacts cannot support.

Build

Change the constant addend or mux selection in the lab RTL, predict the structural effect, and rerun the same lab. Explain the difference without using cell count as a claim about final transistor count or silicon area.

Why? notebook

1. Why can simulation accept constructs that synthesis cannot build?
2. Why is elaboration necessary before optimization?
3. Why can an inferred adder disappear?
4. Why does technology mapping require a target?
5. Why can a netlist still fail timing after routing?
6. Why can pipelining help frequency but hurt latency, area, and control?
7. Which parts of `acc += input` belong to the ISA, microarchitecture, RTL, gate network, and physical device layers?

Next bridge: use this physically grounded model to study CPU microarchitecture, memory translation and privilege, then cross the ISA and exception boundaries while building ToyOS.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Yosys Synthesis Lab

Format: shell lab

What you will do: Observe one small RTL design before and after generic technology mapping.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 04-cpu-and-chip-design/challenges/day-007-yosys-synthesis-lab.sh
```

- View lab source on GitHub